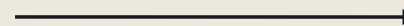


SLEEP-TIME COMPUTE

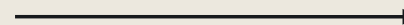
기억 생애주기의 다음 인프라

장기 기억 · continual learning · background training 을 하나의 lifecycle 로 재구성

WAKE



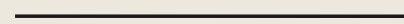
SNAPSHOT



SLEEP



MEMORY



기억 lifecycle 을 새로운 scaling · infra 축으로 읽는다 .

핵심은 ‘자는 모델’ 이 아니라 기억의 이동 규칙이다

통합 추론

언제

배포 전 · wake/test-time · delayed sleep

무엇을

replay · distill · synthesize · merge · verify

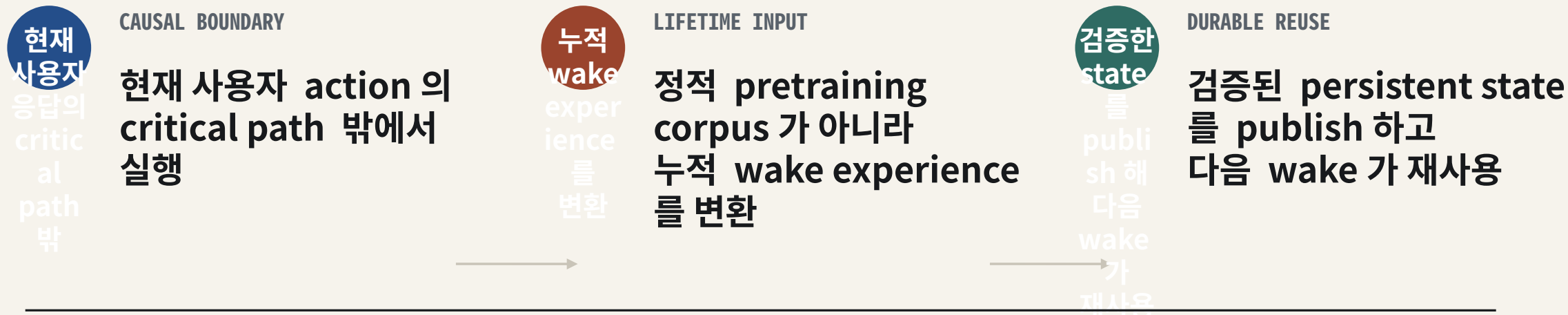
어디로

text/vector/graph → state/adapter → shared weight

모든 이동은 provenance · capacity · rollback · deletion 을 함께 가져야
한다.

Strict sleep 은 세 조건을 동시에 만족한다

정의



CAUSAL BOUNDARY 현재 사용자 응답의 critical path 밖
Idle compute 나 긴 reasoning 만으로는 sleep 이 아니다 .

앞으로의 학습은 세 phase 와 하나의 memory fabric 으로 나뉜다

통합 추론

PRE

pretraining

pretrain · post-train
update rule 학습

WAKE

infer

infer · retrieve
TTT · capture

SLEEP

replay

replay · distill
compact · validate

MEMORY

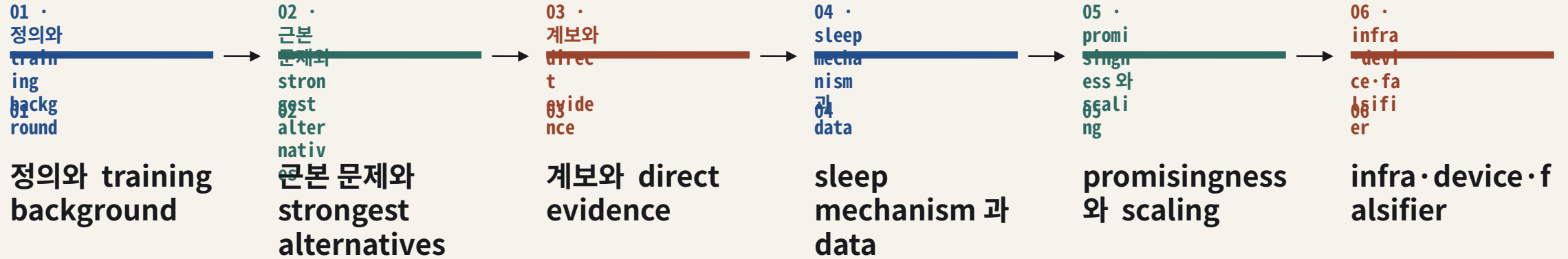
canonical
evid...

canonical evidence
views · adapters

Fixed base 와 session/domain mutable state 를 version contract 로 연결한다 .

세미나는 정의에서 device experiment 까지 한 방향으로 간다

안내



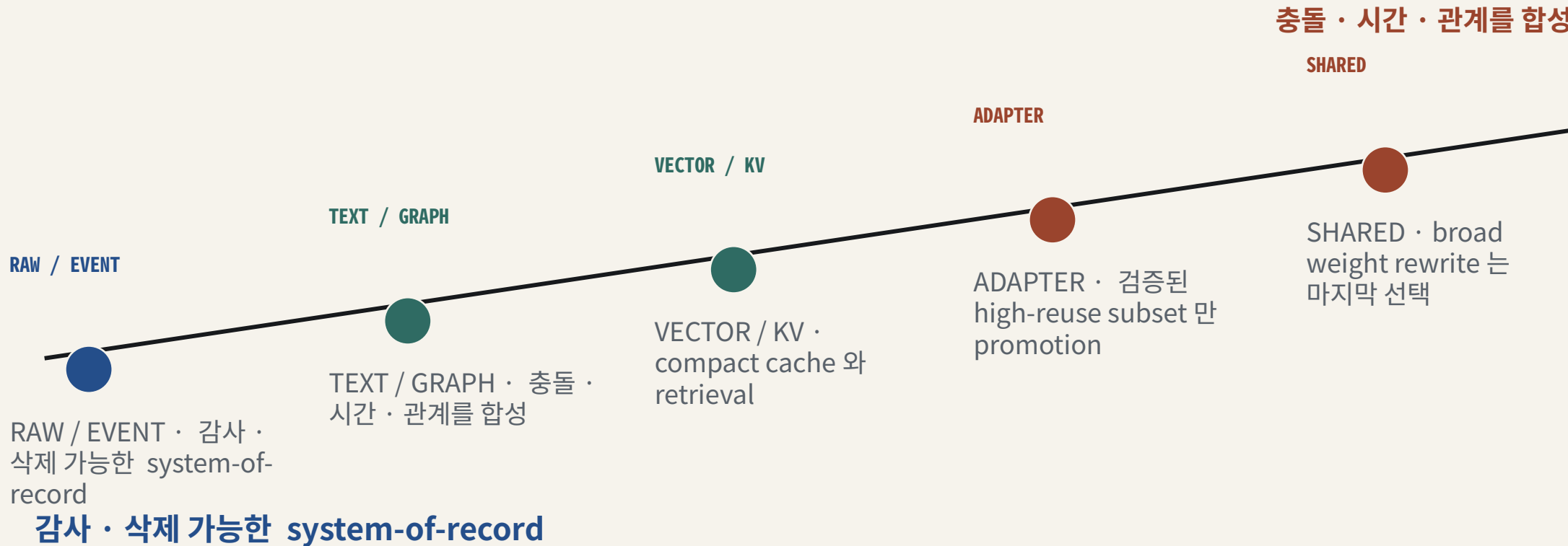
01

답보다 먼저 비교 기준과 반증 조건을 고정한다 .

01

현재의 최선은 external-first, parameter-last hybrid 다

조건부 결론



Reversibility 와 provenance 가 promotion gate 의 첫 조건이다 .

Production 신호는 external sleep 이 강하고 parametric sleep 은 연구 단...

STORAGE
· 외부
episode
· graph
는 제품
실험
작은
continu
al-
ADDRESS
ABILITY
실험은
weight
실험
실패
학습과
보존은
GOVERN
ANCE ·
rollbac
k·delet
e·secur
ity는
미성숙



외부 episode·graph 는 제품 신호가 강함



작은 continual-learning 실험은 직접 증거



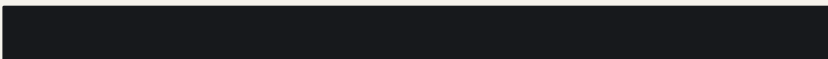
weight 에 남아도 access 실패 가능



새 학습과 보존은 충돌



rollback·delete·security 는 미성숙



STORAGE

오늘의 단위는 ‘검증 가능한 기억 lifecycle’ 이다

안내

CAPTURE

TRANSFORM

VERIFY

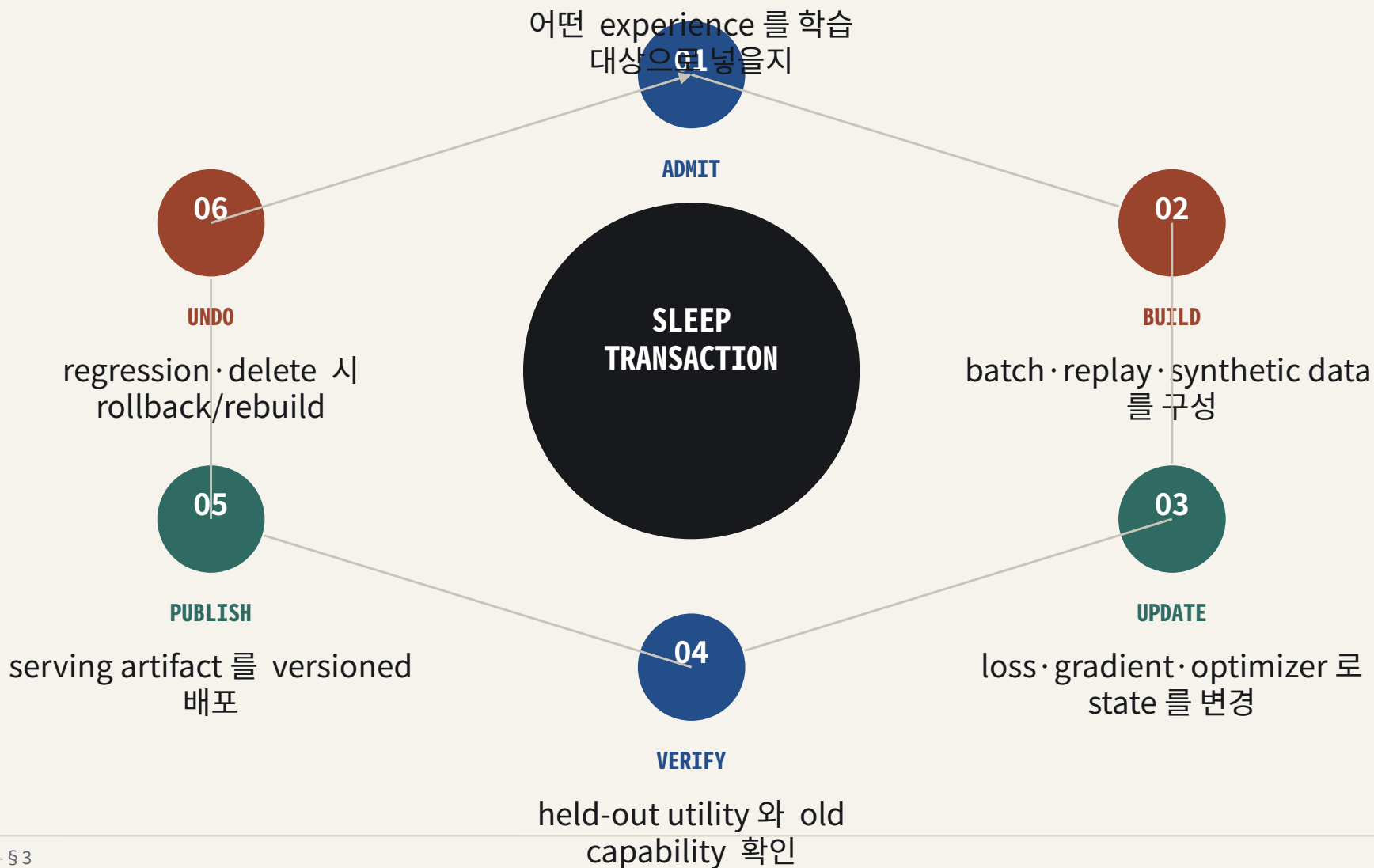
REUSE / UNDO



Wake 가 exact event 를 남기고
Sleep 이 candidate 를 만들며
Memory fabric 이 version 을 보유하고
Control plane 이 publish 와 undo 를 결정한다

원 evidence 와 consent 를 남긴다

Training 은 data 를 미래 행동으로 변환하는 operator pipeline 이다 —



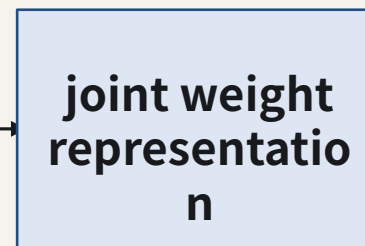
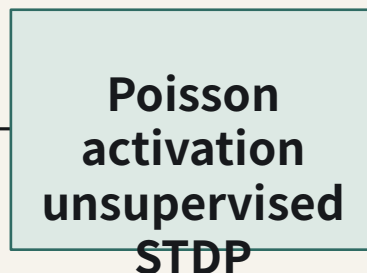
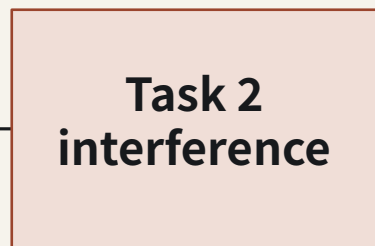
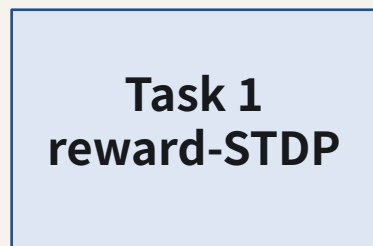
한 번의 update 는 forward→loss→backward→optimizer 순서다

FORWARD

LOSS

BACKWARD

UPDATE



01
neurons
2 tasks
≥10 inits

FORWARD 02

LOSS 03

LIMIT old trace 가 완전히 사라진 뒤에는
복원하지 못했다 .
LLM · 다과제 · 장기용량 ·
deletion 은 검증하지 않았다 .

Objective · metric · evaluator 는 같은 것이 아니다

배경



직접
최적화
하는
수치

OBJECTIVE

현재 사용자 action 의
critical path 밖에서
실행



에서
보고하
는
성공 ·
비용

METRIC

정적 pretraining
corpus 가 아니라
누적 wake experience
를 변환



의
품질
판정자

EVALUATOR

검증된 persistent state
를 publish 하고
다음 wake 가 재사용

OBJEC
TIVE

학습 중 직접 최적화하는 수치

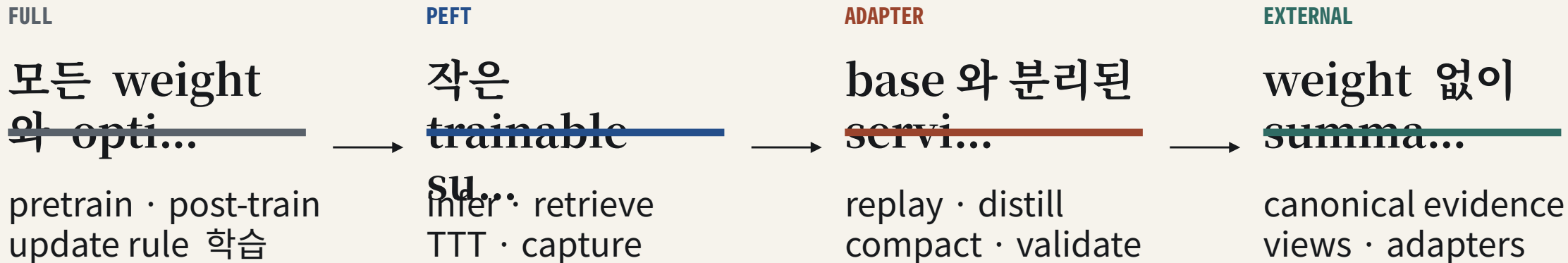
Evaluator 가 틀리면 sleep 은 오류를 durable state 로 승격한다 .

Sleep data 는 real·replay·synthetic·feedback 을 함께 다룬다

	REAL · 원 episode· 문서· tool outcome	REPLAY · old capability 보호 example	SYNTHETIC · dream· counterfactual· hard negative	FEEDBACK · preference· correction· reward
REAL · 원 episode· 문서· tool outcome	REAL	REPLAY	SYNTHETIC	
REPLAY · old capability 보호 example	Letta sleeper Auto-Dreamer	Cartridges Offline Recurrence	REAL	REPLAY
SYNTHETIC · dream· counterfactual· hard negative	memory-policy pretraining		FEEDBACK	PROVENANCE

원 episode· 문서· tool outcome
old capability 보호 example

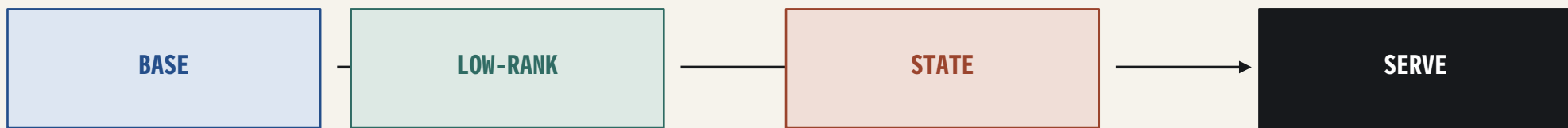
Fine-tuning 은 full weight 부터 external transform 까지 연속체다



Update cost 와 governance 는 destination 에 따라 달라진다 .

LoRA 는 update 를 작게 만들지만 training 을 없애지는 않는다

배경



$$\Delta W = B A, \\ \text{rank}(A, B) \ll d$$



공유
foundation
weight 는

BASE

작은 delta 는 rollback 단위를 만들지만 sequential interference 를 자동 해결하지 않는다 .

$\Delta W=BA$ 로 trainable parameter 축소

Distillation 은 behavior 를 옮기지만 무엇을 버릴지도 결정한다

배경

TEACHER

- 긴 context · fast memory · old model 의 output
- adapter · slow memory · compact state
- logit/behavior divergence 를 최소화
- teacher error 와 rare-tail loss 도 압축

STUDENT

- 긴 context · fast memory · old model 의 output
- adapter · slow memory · compact state
- logit/behavior divergence 를 최소화
- teacher error 와 rare-tail loss 도 압축

SIGNAL

긴 context · fast memory · old model 의 output

Replay 와 augmentation 은 retention data plane 을 만든다

배경

RAW REPLAY

fidelity 는 높지만 storage · privacy 가 큼

CORESET

대표 example 만 남겨 byte 절감

GENERATED

generator 가 old distribution 을 근사

AUGMENT

counterfactual · rare case 를 확장

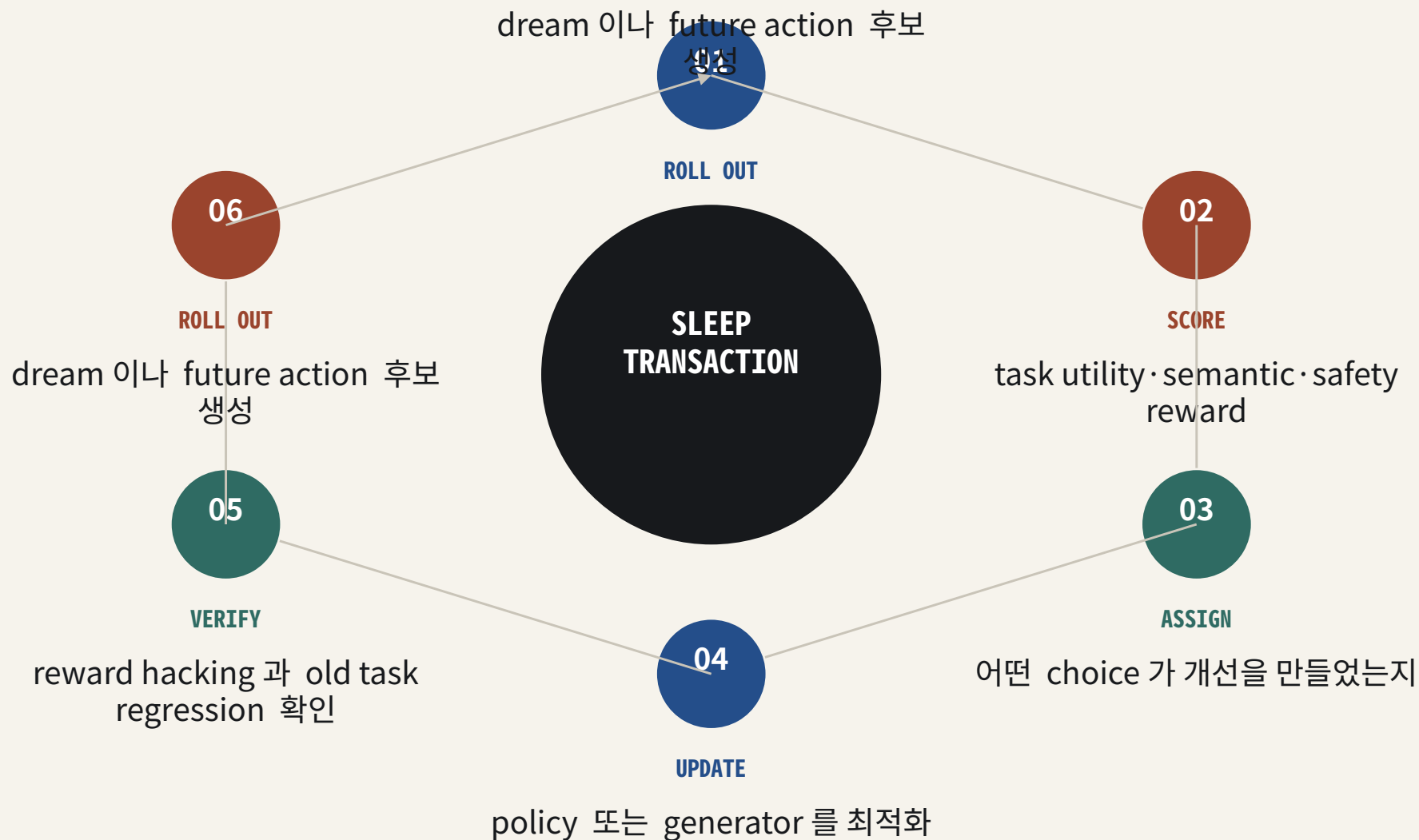
RAW REPLAY

fidelity 는 높지만
storage · privacy 가 큼

fidelity 는 높지만 storage · privacy 가 큼

RL-style sleep 은 rollout→reward→update 의 delayed loop

다 배경 —



Continual learning 은 retention 과 plasticity 를 동시에 요구한다

RETENTION · old trace 를 recall
PLASTICITY · fresh control 만큼 새 task 를 학습
ACCESS · encoded information 을 실제 answer 에 적용
CAPACITY · finite state 에서
GOVERNANCE · source · delete · rollback 을 보존

RETENTION

old trace 를 recall threshold 위에 유지

PLASTICITY

fresh control 만큼 새 task 를 학습

ACCESS

encoded information 을 실제 answer 에 적용

CAPACITY

finite state 에서 interference 를 관리

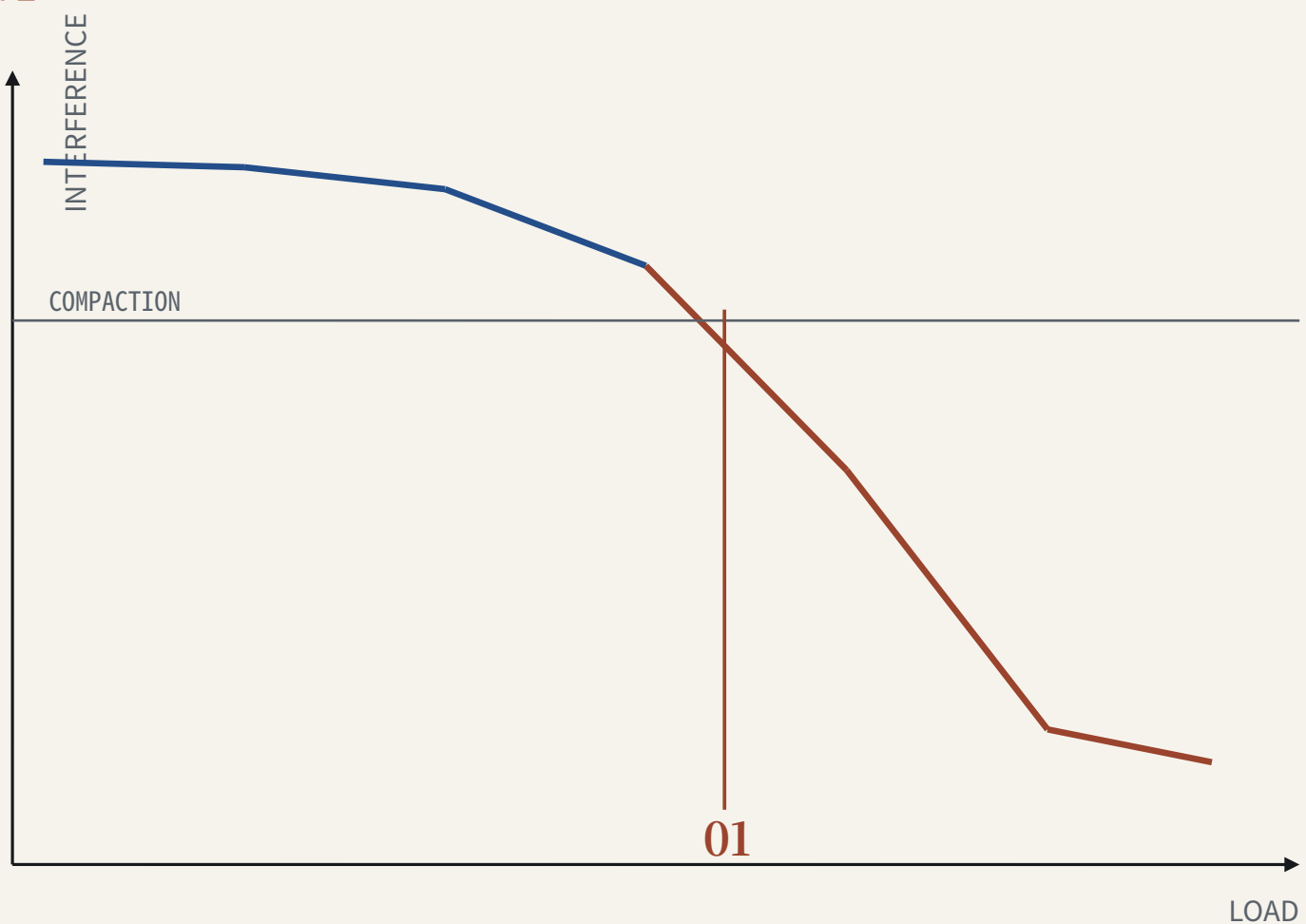
GOVERNANCE

source · delete · rollback 을 보존

RETENTION

Capacity 는 parameter 수보다 utility 가 꺾이는 경계다

통합 추론



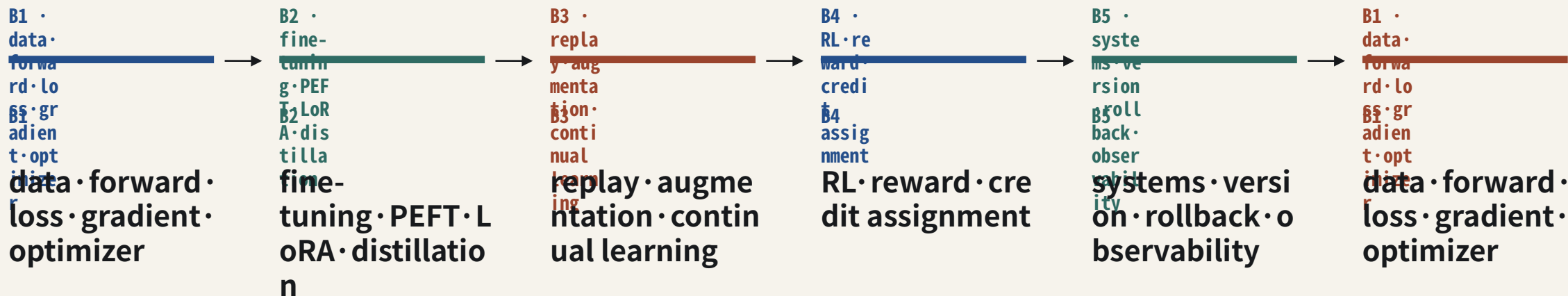
KNEE

- LOAD
- INTERFERENCE
- COMPACTION
- KNEE
- LOAD

Static memorization capacity 는
lifelong safe capacity 가 아니다 .

Background paper 는 mechanism 을 읽기 위한 공통 언어다

안내



B1

모든 training 용어는 PDF named destination 으로 연결되어 있다 .

01

Long context 는 작업대이지 장기기억 lifecycle 이 아니다

문제



Context capacity $\neq$ durable $\longrightarrow$ 선택한 evidence 를 한 request 에서

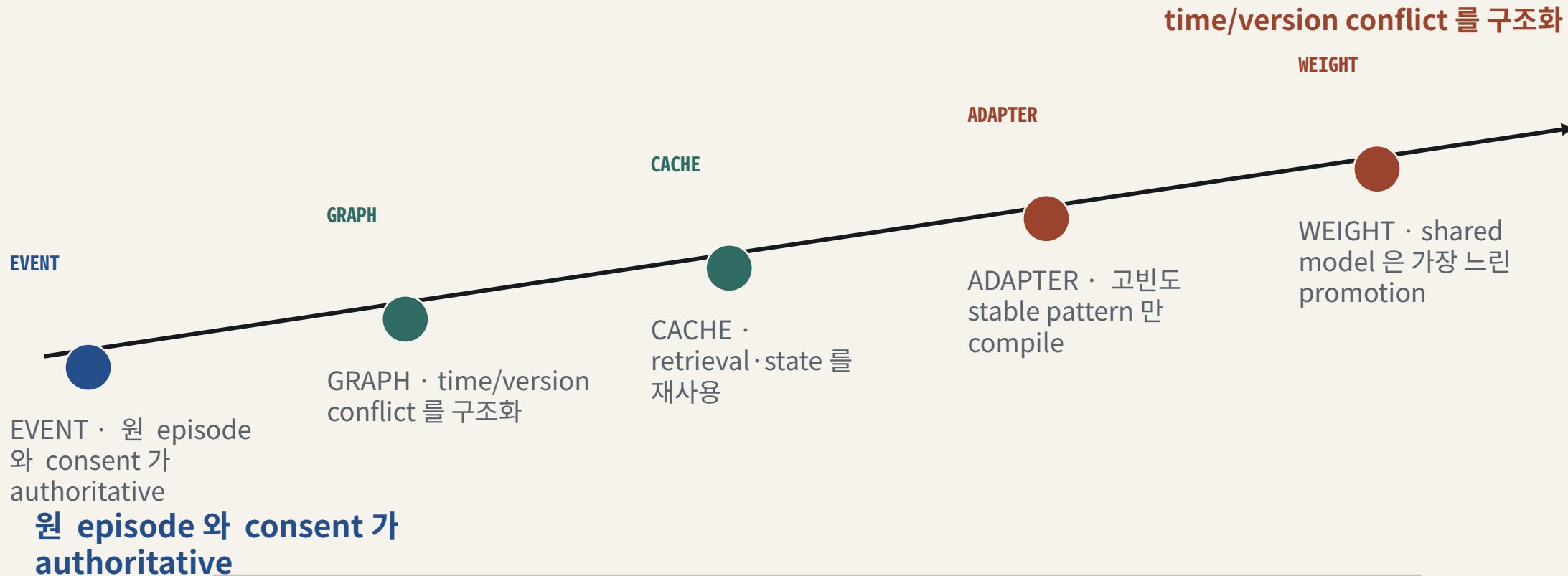
READ

Context 가 커져도 admission · conflict · staleness · deletion 은 남는다 .

durable update · merge · delete 는 별도 문제

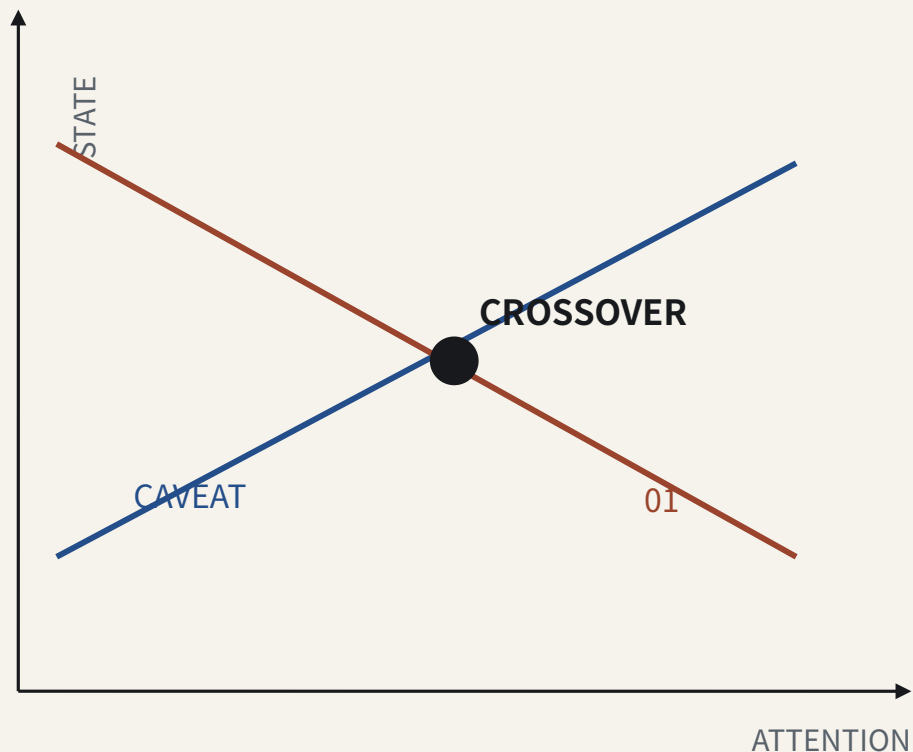
Canonical truth 와 compiled memory 를 분리해야 한다

통합 추론



Derived state 는 source evidence 에서 재생성 가능해야 한다 .

Full attention 의 reuse 비용이 길어질수록 state 압축이 매력적이다



ATTENTION

— prefix load 와 KV 가
sequence 에 따라 증가

STATE

— fixed/learned state 는
prefix 를 압축

CROSSOVER

— reuse 와 sequence
길이가 break-even 결정

CAVEAT

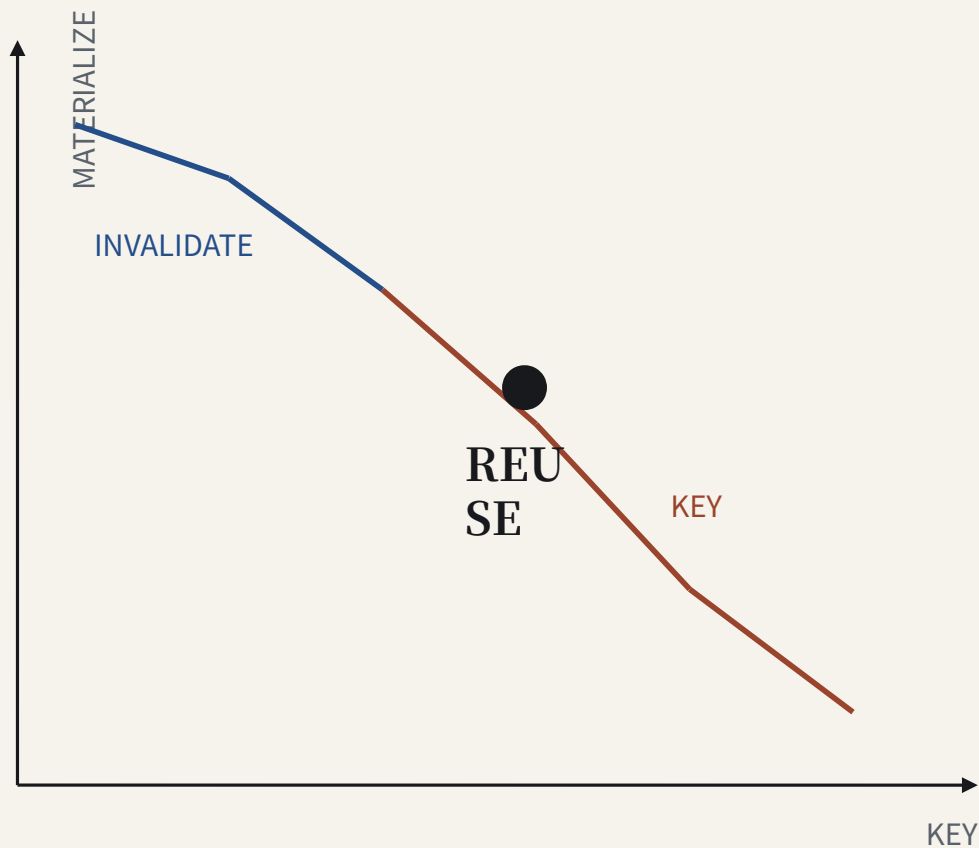
— 압축 fidelity 와 cache
hit 가 필요

ATTENTION

— prefix load 와 KV 가
sequence 에 따라 증가

단일 N 보다 $\text{reuse} \times \text{length} \times \text{state-quality}$ 의
crossover atlas 가 필요하다 .

Recurrent model 도 state cache 가 없으면 serving 이득을 잃는다



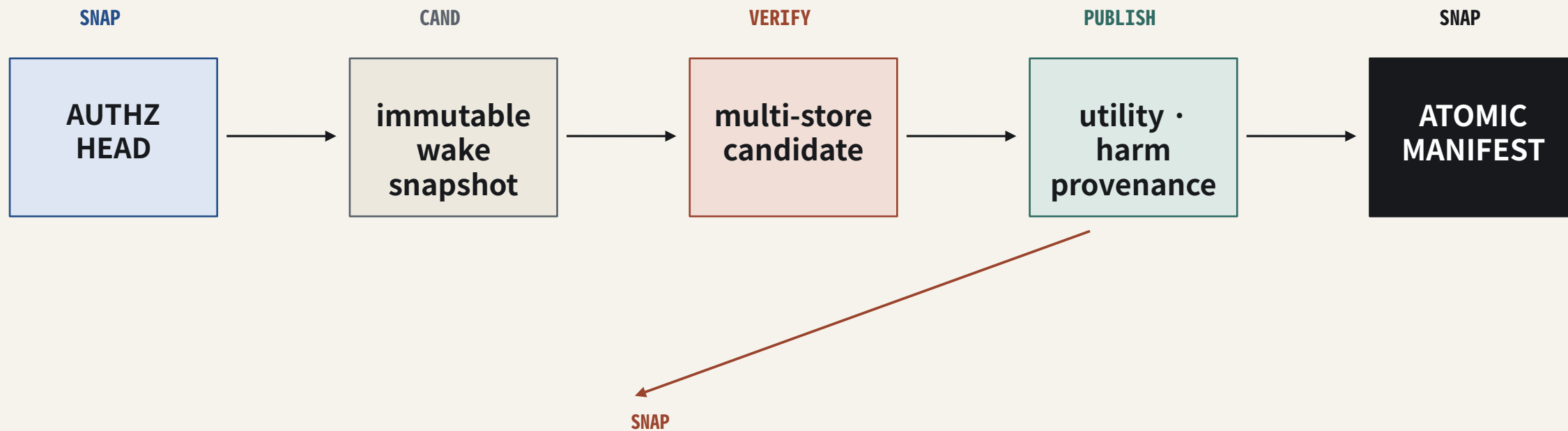
MATERIALIZE

model/adaptor version
+ exact prefix hash

- KEY
- MATERIALIZE
- REUSE
- INVALIDATE

model/adaptor version + exact prefix hash

Stale · conflicting fact 는 update 가 아니라 transaction 문제다



In-place overwrite 보다 immutable generation 과 lineage 가 안전하다 .

In-place overwrite 보다 immutable generation 과 lineage 가 안전하다 .

Memory 는 늘릴수록 좋다가 어느 순간 noise 가 앞선다

가설



AFTER

- EARLY
- MIDDLE
- KNEE
- AFTER
- EARLY

Device byte 와 usable memory capacity 를 같은 값으로 보지 않는다 .

Poison·privacy·delete 는 memory medium 마다 다른 잔여 위험을 남긴다

04

adversarial findings

statistics
capacity
systems
publication

POISON

→ 오염된 record 가 derived state 로 확산

PRIVACY

→ raw replay 와 user delta 가 sensitive data 를 보존

DELETE

→ summary·graph·cache·adapter 까지 전파

RECOVER

→ lineage-driven invalidate 와 rebuild

POISON

→ 오염된 record 가 derived state 로 확산

삭제가 source DB 에서 끝나면 sleep lifecycle 은 불완전하다 .

Research question 은 여섯 개의 answerable 축으로 분해된다

질문

Q1

- Q1
- Q2
- Q3

Q2

- Q1
- Q2
- Q3

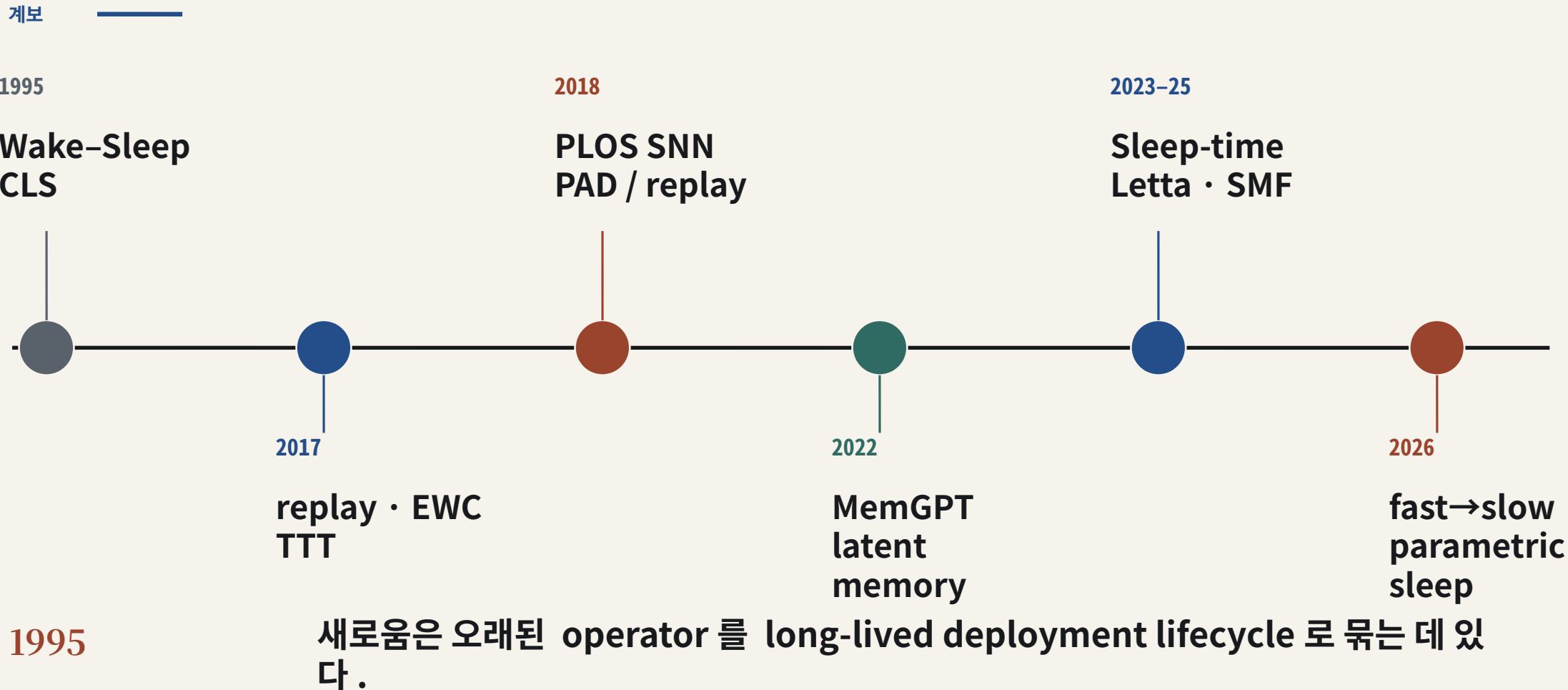
Q3

- Q1
- Q2
- Q3

Q1

각 질문은 strongest baseline 과 falsifier 를 가져야 한다 .

‘Sleep’이라는 이름보다 offline consolidation 계보가 먼저였다



Complementary learning systems 는 fast 와 slow memory 역할을 나눈다

FAST

- episode 를 빠르게 포착하는 hippocampal 역할
- 여러 episode 에서 stable structure 를 통합
- offline reactivation 으로 두 system 연결
- biological plausibility 는 fleet economics 가 아님

SLOW

- episode 를 빠르게 포착하는 hippocampal 역할
- 여러 episode 에서 stable structure 를 통합
- offline reactivation 으로 두 system 연결
- biological plausibility 는 fleet economics 가 아님

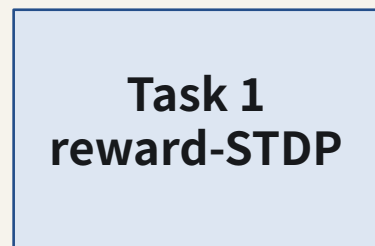
REPLAY

episode 를 빠르게 포착하는 hippocampal 역할

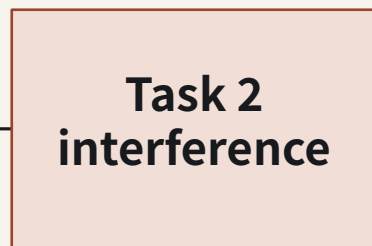
Wake-Sleep 은 generative model 학습 알고리즘에서 시작했다

계보

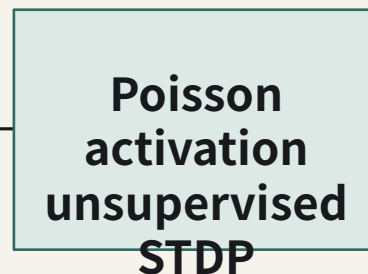
WAKE



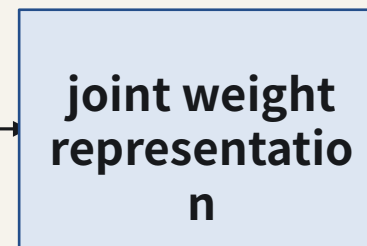
SLEEP



LEGACY



LESSON



01

neurons
2 tasks
 ≥ 10 inits

WAKE

02

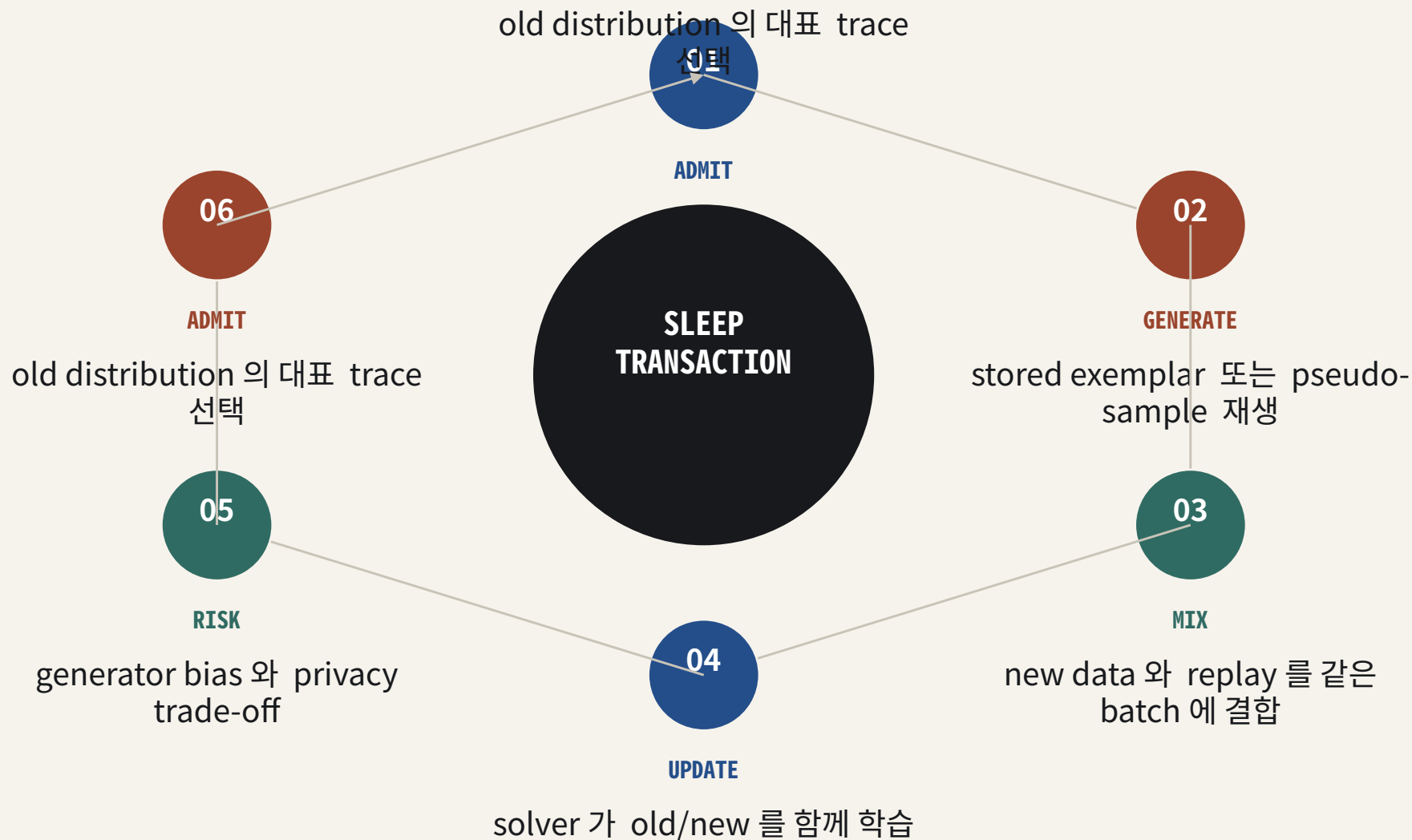
SLEEP

03

LIMIT

old trace 가 완전히 사라진 뒤에는
복원하지 못했다 .
LLM · 다과제 · 장기용량 ·
deletion 은 검증하지 않았다 .

Pseudorehearsal 과 generative replay 는 old data 를 재구성한다



EWC 와 GEM 은 sleep 없이 update 충돌을 줄인다

대안

EWC old-important parameter 이동에
Fisher penalty

GEM old example loss 가 증가하지
않도록 gradient projection

STRENGTH retention 문제에 직접 대응

task 증가 · buffer · governance debt

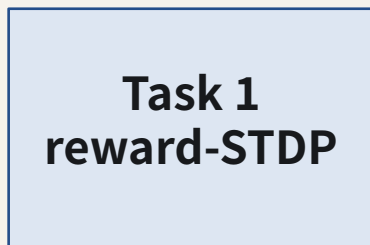
LIMIT old-important parameter
이동에 Fisher penalty

- EWC
- GEM
- STRENGTH
- LIMIT

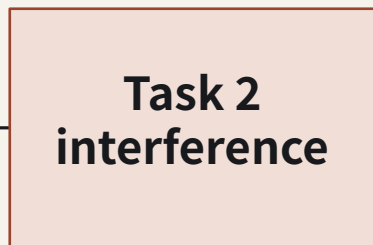
Sleep 은 이 baseline 보다 lifecycle utility 에서 이겨야
한다 .

Progress & Compress 는 active learning 과 consolidation 을 고대했다

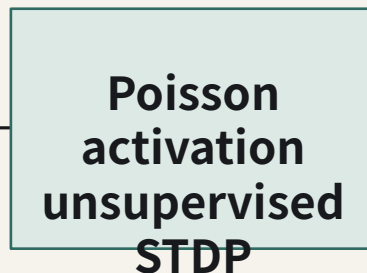
ACTIVE



PROGRESS



COMPRESS



DIRECT LINK



01

neurons
2 tasks
≥10 inits

ACTIVE

02

PROGRESS

03

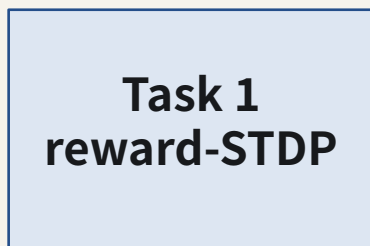
LIMIT

old trace 가 완전히 사라진 뒤에는
복원하지 못했다 .
LLM · 다과제 · 장기용량 ·
deletion 은 검증하지 않았다 .

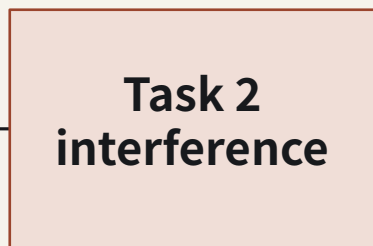
2022 PLOS 는 두 과제 SNN 에서 sleep-like replay 를 검증했다

직접 근거

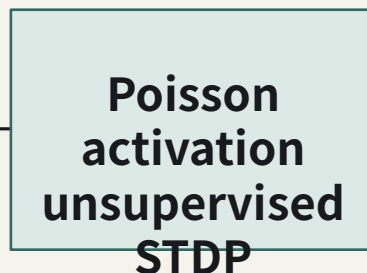
WAKE 1



WAKE 2



SLEEP



LATER



842

neurons
2 tasks
≥10 inits

WAKE 1

2

WAKE 2

≥10

LIMIT

old trace 가 완전히 사라진 뒤에는
복원하지 못했다 .
LLM · 다과제 · 장기용량 ·
deletion 은 검증하지 않았다 .

NREM/REM alternation 은 integration 과 protection objective 를...

NREM

- replay-like autonomous dynamics 로 새 정보 통합
- older cortical knowledge 와의 관계 재구성
- phase alternation 이 interference 를 조절
- model scale 과 task abstraction 이 제한적

REM

- replay-like autonomous dynamics 로 새 정보 통합
- older cortical knowledge 와의 관계 재구성
- phase alternation 이 interference 를 조절
- model scale 과 task abstraction 이 제한적

INTERLEAVE

replay-like autonomous dynamics 로 새 정보 통합

PAD 는 wake · perturbed replay · adversarial dreaming 을 분리했다

WAKE episodic latent reconstruction

E

NREM-LIKE noise · occlusion 으로 robust replay

REM-LIKE adversarial dream 으로 representation 확장

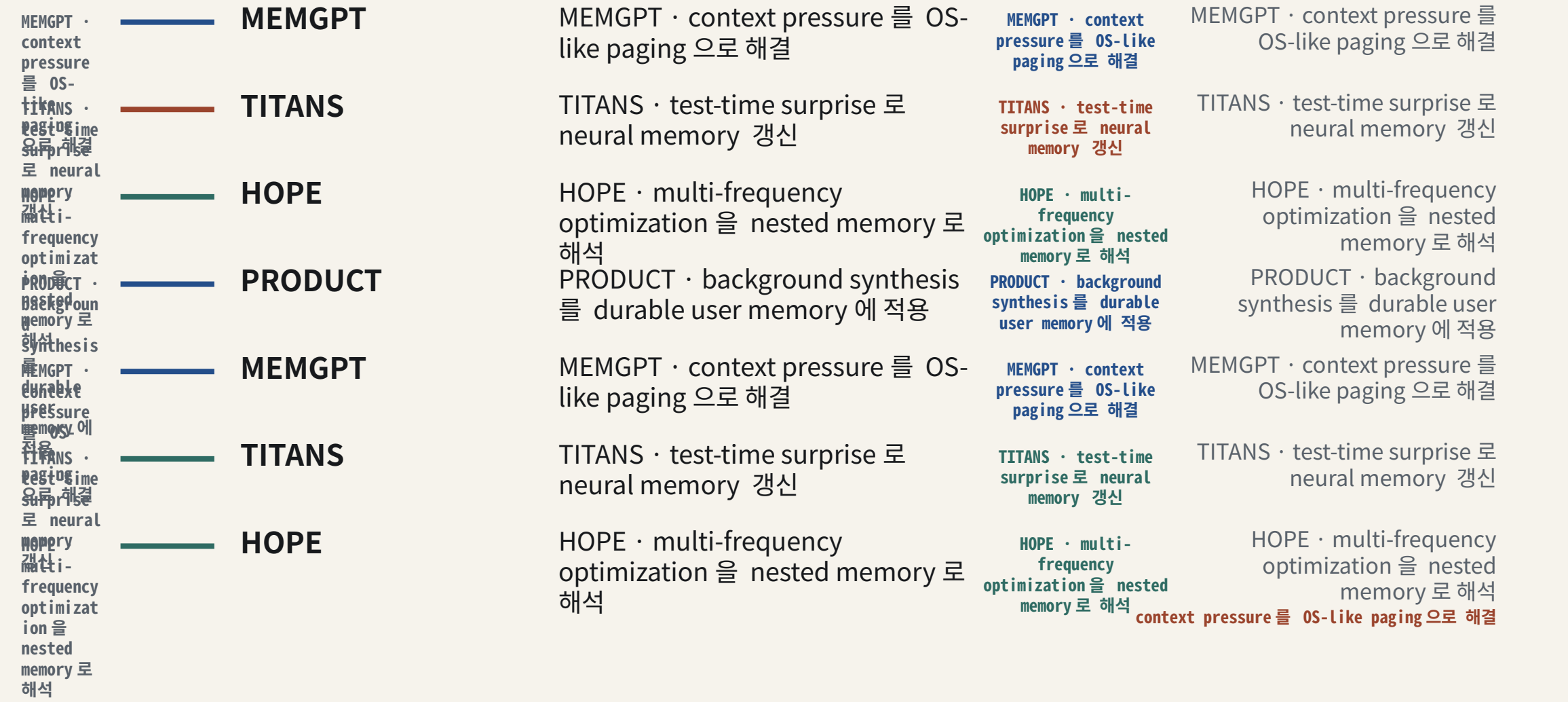
phase-specific operator 후보

VALUE modern LLM system evidence
는 아님

- WAKE
- NREM-LIKE
- REM-LIKE
- VALUE

Biological label 은 mechanism evidence 를 대체하지 않는다 .

계보는 agent external memory 와 neural memory 에서 합류한다



문제마다 sleep 의 strongest alternative 가 다르다

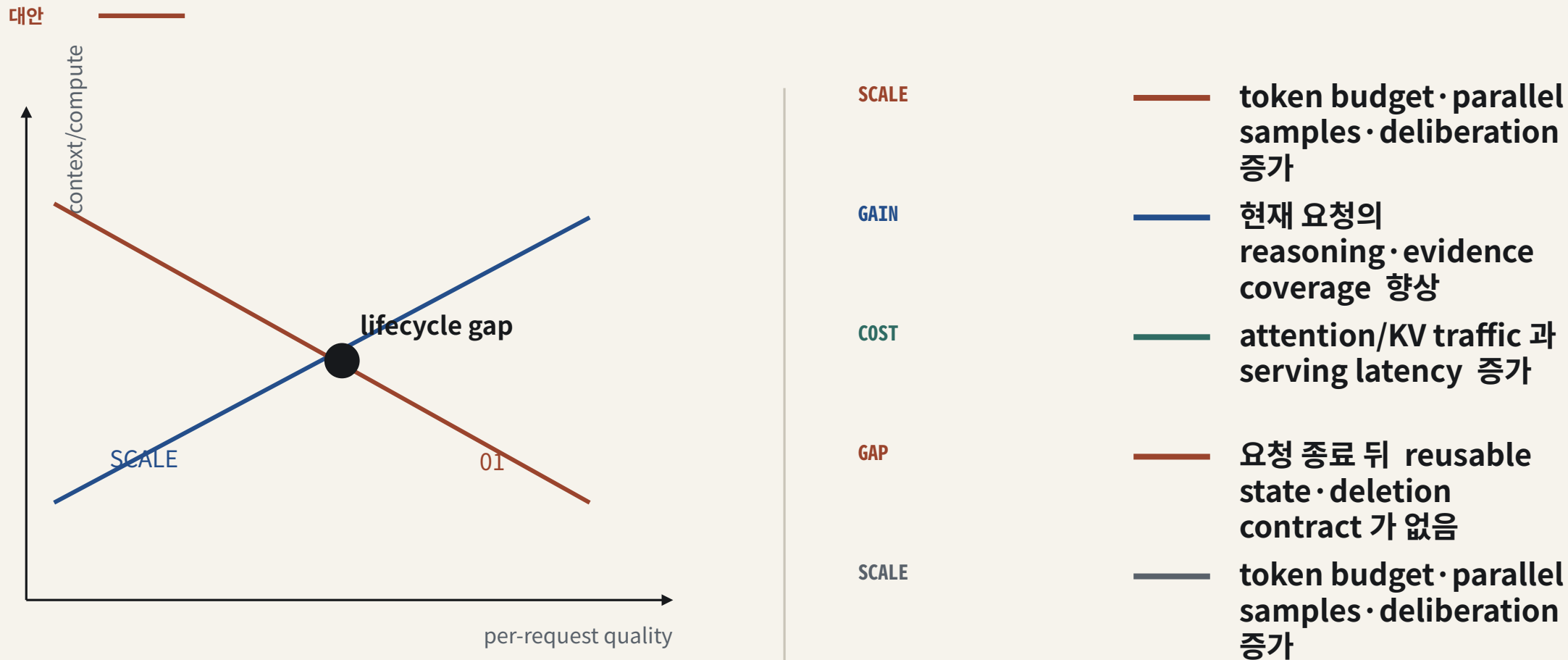
비교

	ACCESS · RAG · long context · recurrent state	RETENTION · replay · regularization · modular growth	PERSONALIZE · prompt/prefix · adapter · external profile	COMPACT · summary · graph merge · distillation
ACCESS · RAG · long context · recurrent state	ACCESS	RETENTION	PERSONALIZE	
RETENTION · replay · regularization · modular growth	Letta sleeper Auto-Dreamer	Cartridges Offline Recurrence	ACCESS	RETENTION
PERSONALIZE · prompt/prefix · adapter · external profile	memory-policy pretraining		COMPACT	GOVERN

RAG · long context · recurrent state

replay · regularization · modular growth

더 긴 inference 가 memory lifecycle 을 대체하지는 않는다



Inference scaling 은 wake capability 이고 sleep 은 cross-request reuse 문제다 .

Prompt compression 은 가장 싼 ‘sleep-like’ baseline 이다

강한 대안

INPUT

- 긴 대화 · 문서를 compact representation 으로 변환
- 다음 wake 의 context 에 다시 삽입
- weight update 없이 cheap·reversible
- 세부 손실 · summary drift·citation 단절

OUTPUT

- 긴 대화 · 문서를 compact representation 으로 변환
- 다음 wake 의 context 에 다시 삽입
- weight update 없이 cheap·reversible
- 세부 손실 · summary drift·citation 단절

STRENGTH

긴 대화 · 문서를 compact representation 으로 변환

Retrieval 은 canonical truth 와 reusable access 를 분리한다

강한 대안

STORE · 원문 · episode 를 audit 가능한 형태로 보존
vector · keyword · metadata 로 candidate 생성
RERANK · 현재 query 와 policy 로 selection bounded context 또는 tool result 로 제공
UPDATE · delete · correct · reindex 가 weight 보다 직접적
STORE · 원문 · episode 를 audit 가능한 형태로 보존
vector · keyword · metadata 로 candidate 생성

STORE

INDEX

RERANK

INJECT

UPDATE

STORE

INDEX

STORE · 원문 · episode 를 audit 가능한 형태로 보존

INDEX · vector · keyword · metadata 로 candidate 생성
RERANK · 현재 query 와 policy 로 selection

INJECT · bounded context 또는 tool result 로 제공

UPDATE · delete · correct · reindex 가 weight 보다 직접적

STORE · 원문 · episode 를 audit 가능한 형태로 보존

INDEX · vector · keyword · metadata 로 candidate 생성

STORE · 원문 · episode 를 audit 가능한 형태로 보존

INDEX · vector · keyword · metadata 로 candidate 생성

RERANK · 현재 query 와 policy 로 selection

INJECT · bounded context 또는 tool result 로 제공

UPDATE · delete · correct · reindex 가 weight 보다 직접적

STORE · 원문 · episode 를 audit 가능한 형태로 보존

INDEX · vector · keyword · metadata 로 candidate 생성

STORE · 원문 · episode 를 audit 가능한 형태로 보존

INDEX · vector · keyword · metadata 로 candidate 생성

RERANK · 현재 query 와 policy 로 selection

INJECT · bounded context 또는 tool result 로 제공

UPDATE · delete · correct · reindex 가 weight 보다 직접적

STORE · 원문 · episode 를 audit 가능한 형태로 보존

INDEX · vector · keyword · metadata 로 candidate 생성

원문 · episode 를 audit 가능한 형태로 보존

Graph memory 는 관계 · 시간 · 충돌을 명시적으로 보존한다

강한 대안

ENTITY

사람 · 조직 · artifact 를 stable node 로

RELATION

사건 · 근거 · causal link 를 edge 로

TIME

valid-from/to 와 episode sequence

CONFLICT

상충 assertion 을 삭제 대신 병존

QUERY

multi-hop retrieval 과
provenance trace

사람 · 조직 · artifact 를 stable node 로

KV · prefix · recurrent state 는 재사용 단위가 서로 다르다

시스템

KV CACHE

KV CACHE

KV CACHE

Serving 효율은
state reuse ratio
와 invalidation
granularity 로
비교해야 한다 .

PREFIX

PREFIX

PREFIX

Serving 효율은
state reuse ratio
와 invalidation
granularity 로
비교해야 한다 .

RNN STATE

RNN STATE

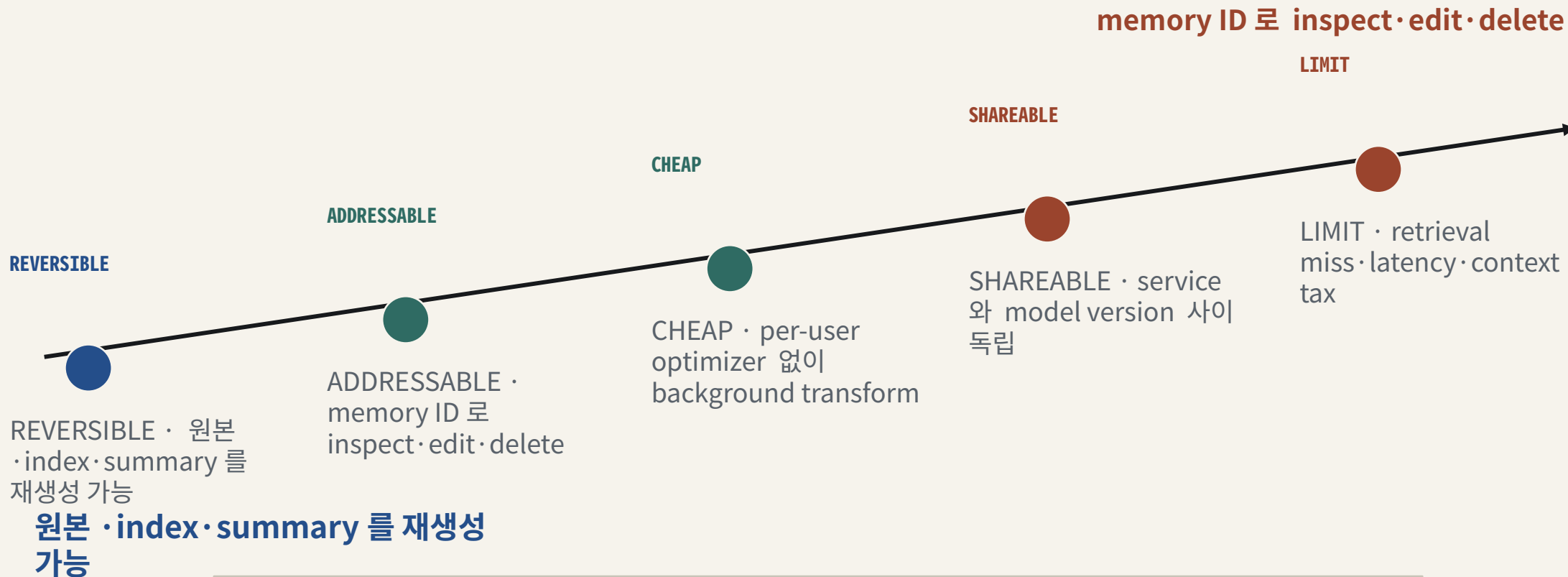
RNN STATE

Serving 효율은
state reuse ratio
와 invalidation
granularity 로
비교해야 한다 .

exact prefix 의 layer activation 재사용

External-first 가 현재 production default 인 이유

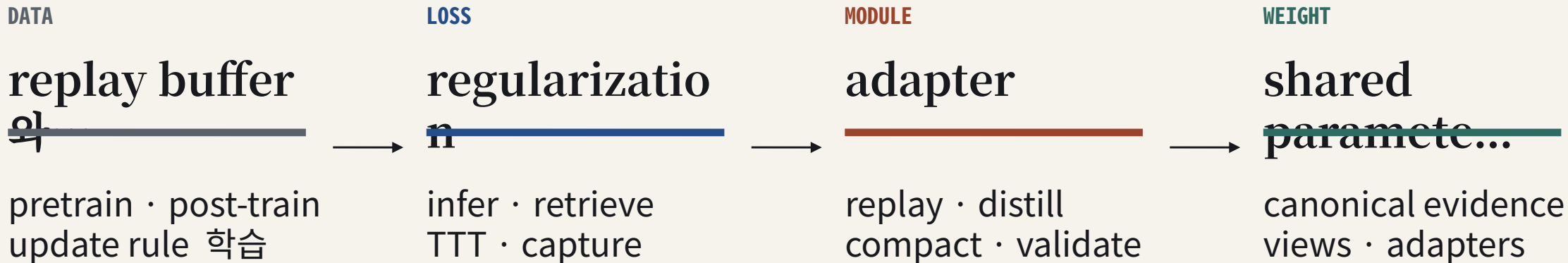
판정



External memory 의 약점은 분명하지만 operational contract 가 성숙하다 .

Continual learning 은 update 위치를 네 층으로 나눈다

대안



Sleep 의 새로움은 technique 보다 이들을 response 밖 lifecycle 로 묶는 데 있다 .

TTT·neural memory 는 wake 에서 state 를 즉시 바꾼다

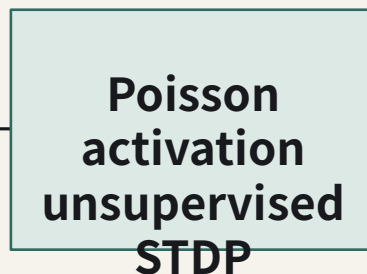
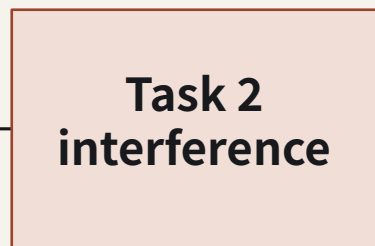
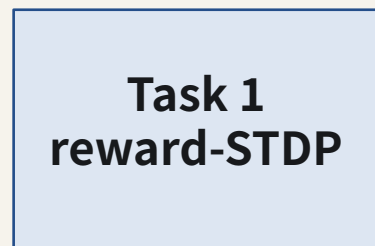
직접 근거

SIGNAL

UPDATE

USE

STRENGTH



01

neurons
2 tasks
≥10 inits

SIGNAL

02

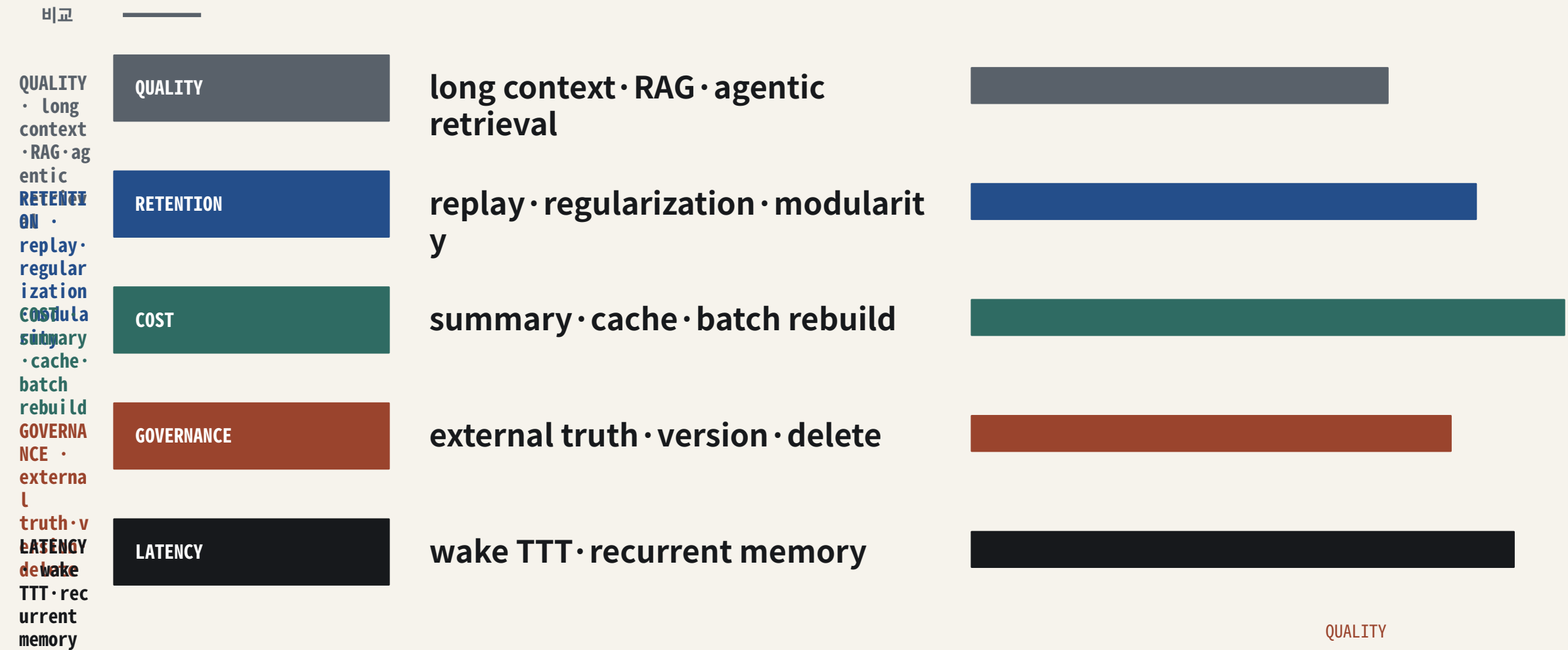
UPDATE

03

LIMI
T

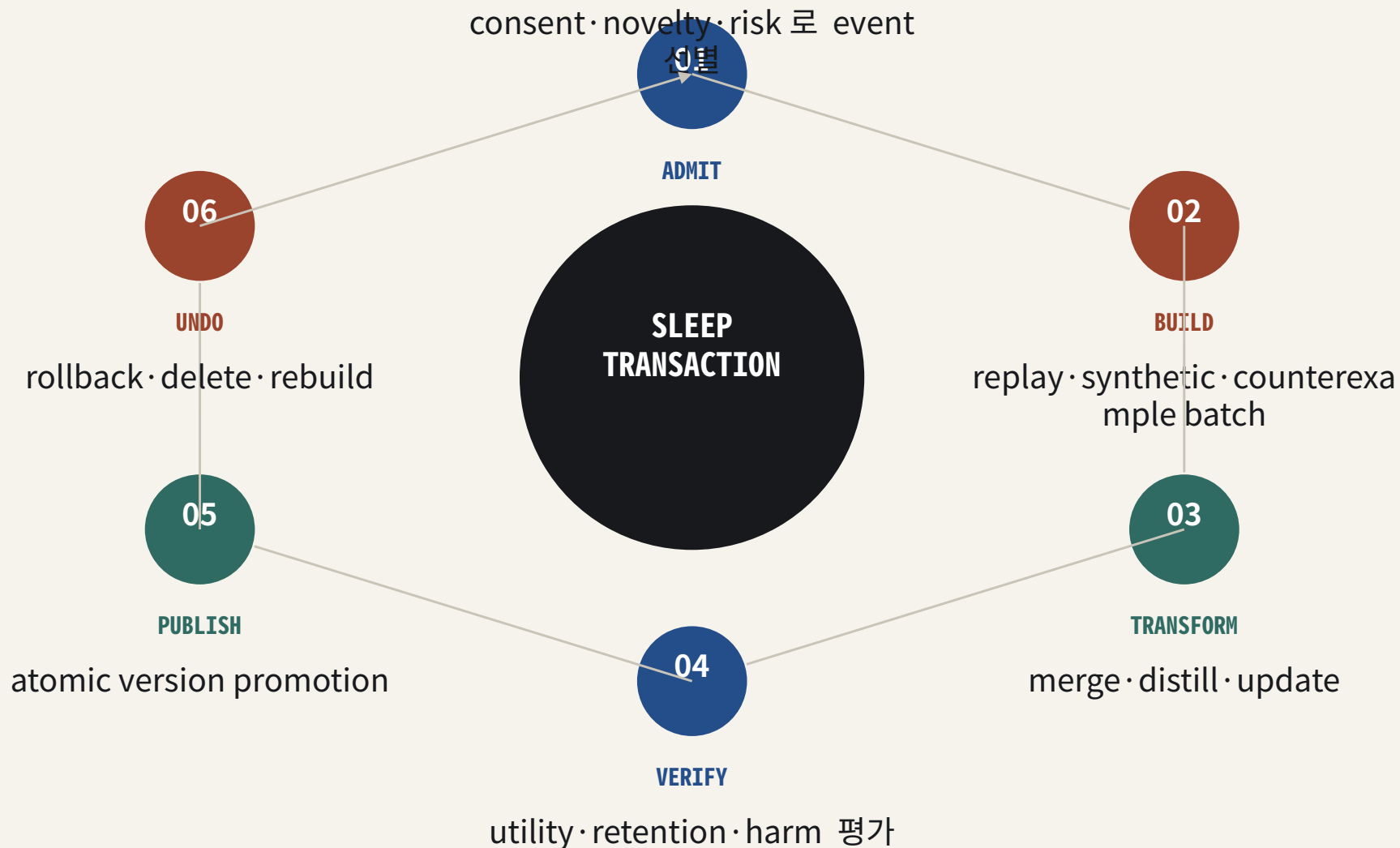
old trace 가 완전히 사라진 뒤에는
복원하지 못했다 .
LLM · 다과제 · 장기용량 ·
deletion 은 검증하지 않았다 .

Sleep 이 이겨야 할 frontier 는 이미 강하다



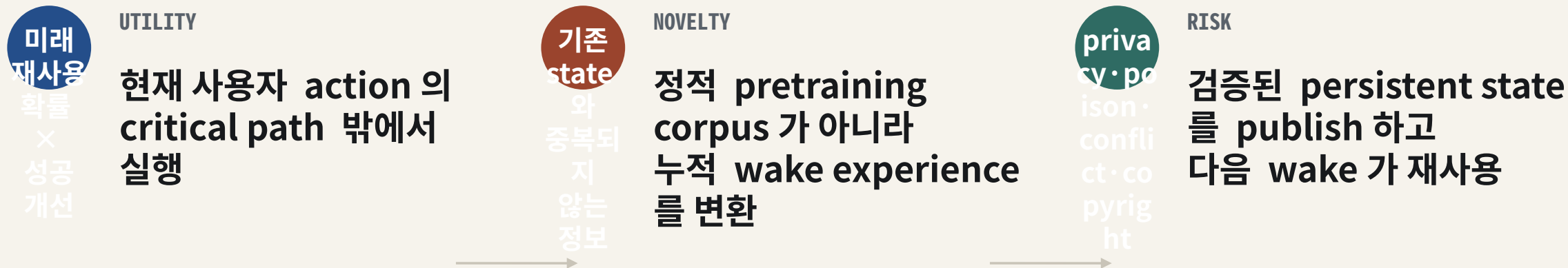
Sleep pipeline 은 여섯 operator 와 두 gate 로 구성된다

제안



Admission 은 ‘많이 저장’ 이 아니라 expected value 판단이다

제안



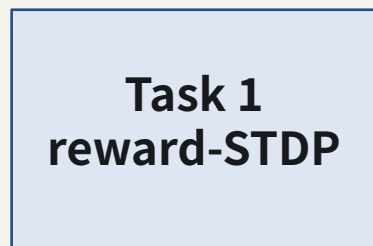
UTILITY 미래 재사용 확률 × 성공 개선

낮은 reuse 와 높은 risk 의 기억은 durable layer 로 승격하지 않는다 .

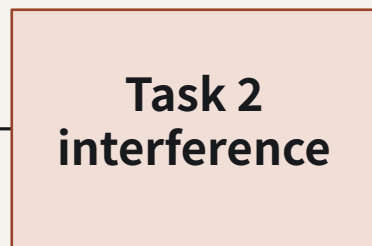
Replay batch 는 new · old · negative 를 함께 담아야 한다

학습

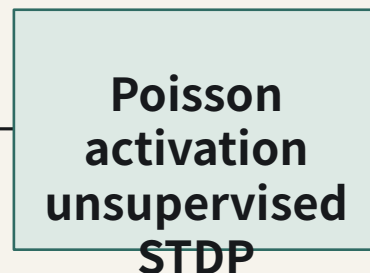
NEW



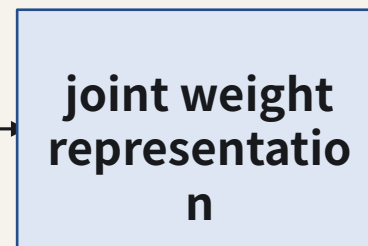
OLD



NEGATIVE



MIX



01

neurons
2 tasks
≥10 inits

NEW

02

OLD

03

LIMIT

old trace 가 완전히 사라진 뒤에는
복원하지 못했다 .
LLM · 다과제 · 장기용량 ·
deletion 은 검증하지 않았다 .

Distillation 은 memory 를 줄이지만 entropy 를 공짜로 없애지 못한다



$$\min L_{\text{task}} + \alpha L_{\text{retain}} + \beta C_{\text{state}} \longrightarrow \text{원} \text{ episode+retrieval+larger}$$

TEACHER

Compression ratio 는 accuracy 뿐 아니라 reversibility 와 deletion cost 로 제한된다 .

summary · adapter · compact state

Dream data 는 coverage 를 늘리지만 오류도 증폭할 수 있다

학습

PERTURB

noise · occlusion · counterfactual 로 robustness

GENERATE

희소 scenario · hard negative 보충

CRITIQUE

generator 와 evaluator 를 분리

grounding · novelty · safety gate

FILTER

model collapse · reward hacking · hallucinated memory

- PERTURB
- GENERATE
- CRITIQUE
- FILTER

Dream 은 truth source 가 아니라 검증 전 candidate data 다 .

Promotion 은 utility · retention · harm 세 축을 동시에 통과해야 한다 —

$+\Delta U$

UTILITY



$\leq \epsilon$

UTILITY

6 deployable $\times$ 3 reuse $\times$ 3 pressure $\times$
3 seeds = 162
+ oracle on 27 operating cells

0

RETENTION

2 schedules $\times$ 2 operators $\times$ 3 reuse
 $\times$ 3 pressure $\times$ 3 seeds

$+\Delta U$

fixed blocks
16 families $\times$
3 seeds

$\leq$

budget

global rows
104
continuous +
27 RF

0

critical

blocked paired
bootstrap

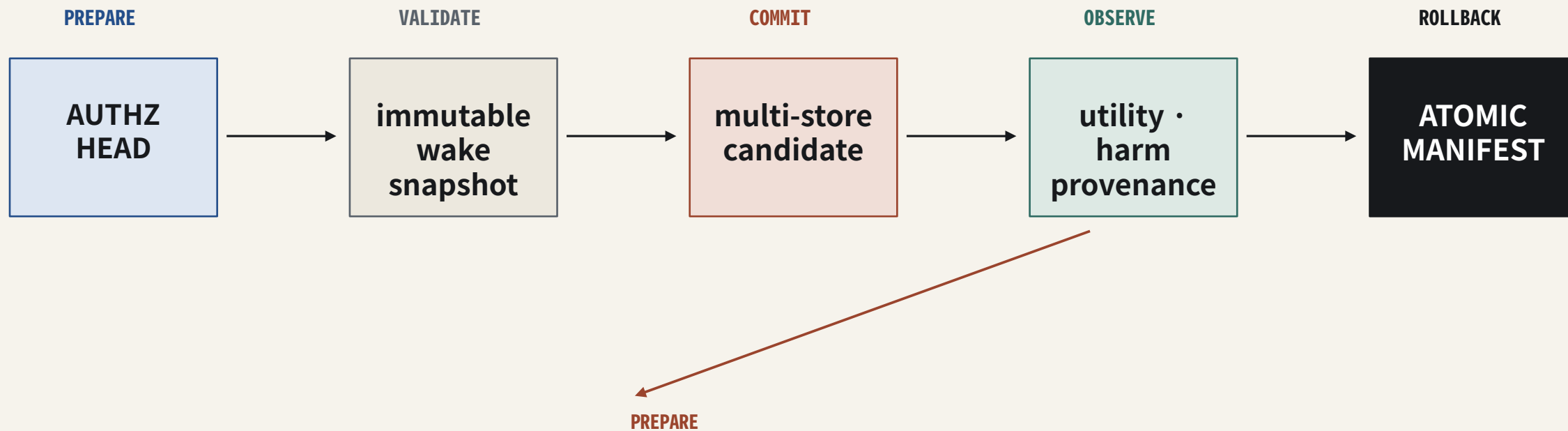
reported

joule · HBM
byte · wall
time · storage

$+\Delta U$

Publish 는 파일 복사가 아니라 state transaction 이다

시스템



Mutable memory 는 model binary 와 같은 release discipline 이 필요하다 .

Mutable memory 는 model binary 와 같은 release discipline 이 필요하다 .

Parametric destination 은 update 반경으로 구분한다

설계

FAST STATE

- 한 sequence·session 안에서만 유효
- 개인 scope 와 명시적 version
- 많은 사용자에서 검증된 reusable delta
- 넓은 blast radius 의 마지막 승격

USER ADAPTER

- 한 sequence·session 안에서만 유효
- 개인 scope 와 명시적 version
- 많은 사용자에서 검증된 reusable delta
- 넓은 blast radius 의 마지막 승격

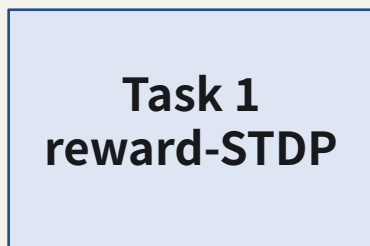
DOMAIN MODULE

한 sequence·session 안에서만 유효

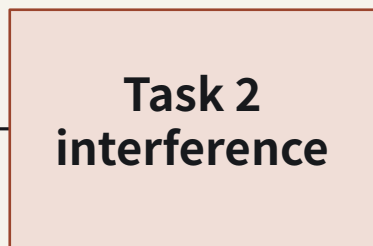
LoRA promotion에는 utility 외 네 가지 gate가 더 필요하다

제안

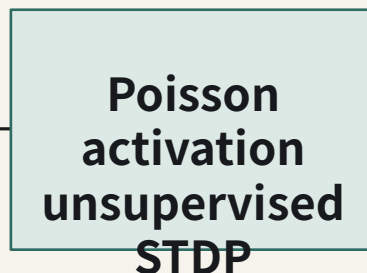
RETENTION



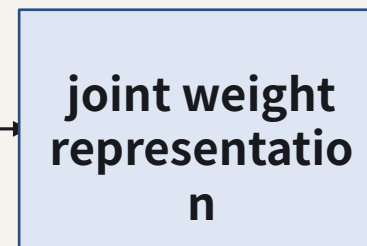
ROUTING



COMPAT



DELETE



01

neurons
2 tasks
≥10 inits

RETENTION 02

ROUTING 03

LIMIT old trace가 완전히 사라진 뒤에는
복원하지 못했다.
LLM · 다과제 · 장기용량 ·
deletion은 검증하지 않았다.

Shared-weight sleep 은 가장 강한 반증을 요구한다

Red team

05

adversarial findings

statistics
capacity
systems
publication

INTERFERENCE

→ unrelated capability 가 조용히 변함

UNLEARNING

→ 한 사용자의 삭제가 global rebuild 로 확대

POISON

→ 작은 malicious stream 이 durable bias 생성

IDENTITY

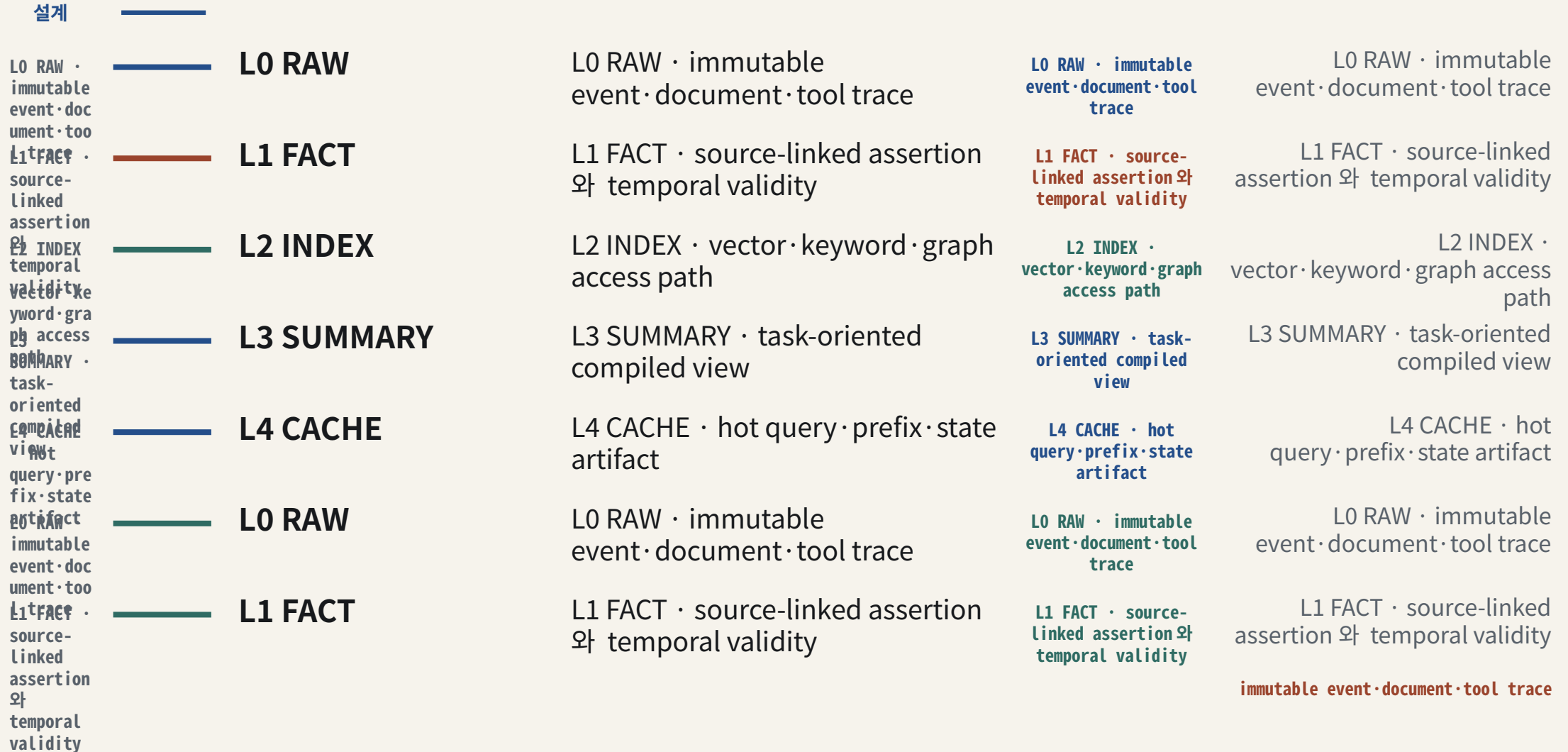
→ 개인 preference 가 shared policy 와 충돌

ECONOMICS

→ optimizer · validation cost 가 reuse gain 초과

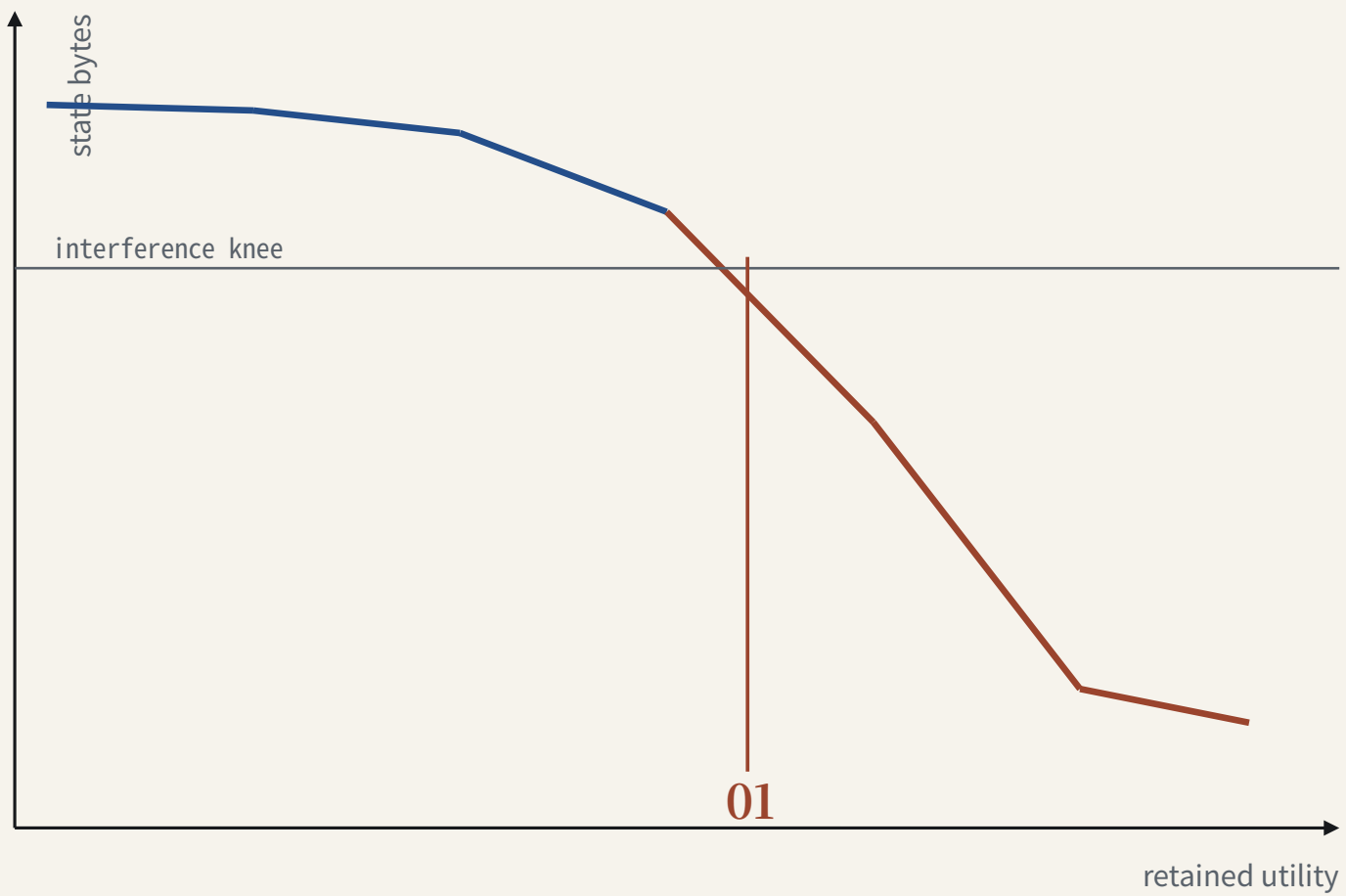
Per-user foundation-weight rewrite 를 기본값으로 두지 않는다 .

External destination 도 raw→derived 계층을 분리해야 한다



Capacity 는 가득 찬 뒤가 아니라 knee 전에 관리한다

이론



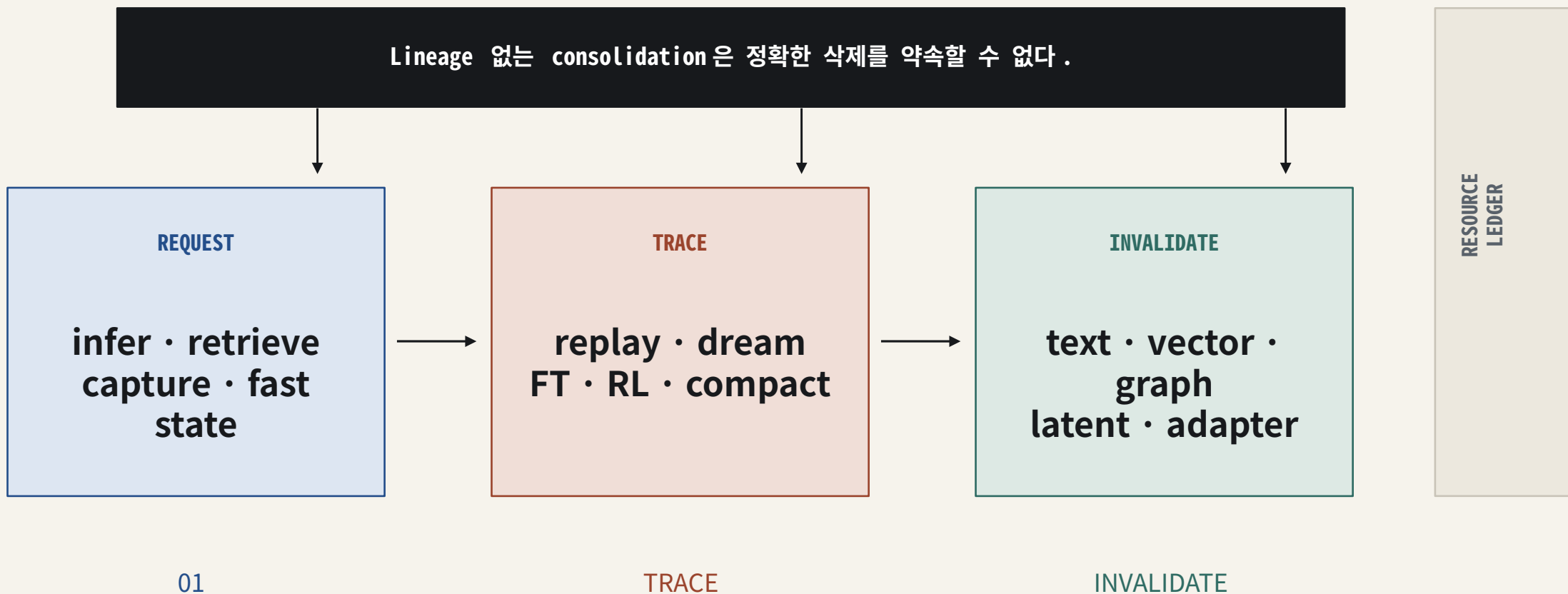
ADMIT

- ADMIT
- DEDUP
- COMPRESS
- EVICT
- REBUILD

용량 문제의 해법은 무한 weight 가 아니라 value-aware lifecycle 이다 .

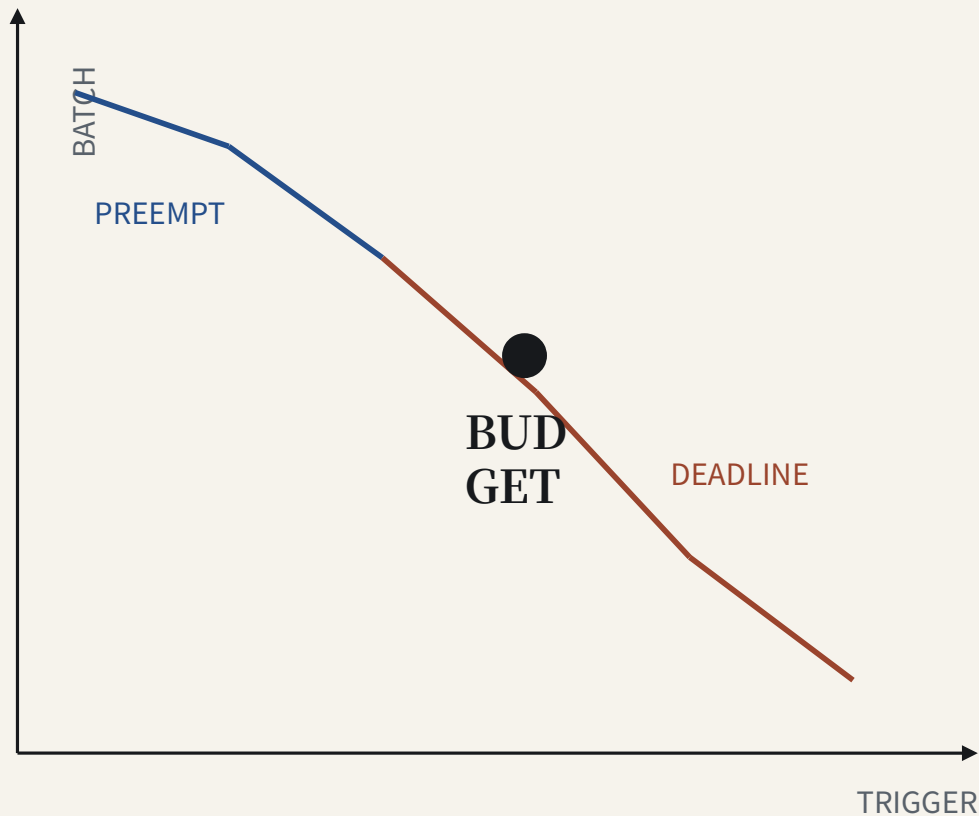
Delete 는 모든 derived state 를 따라가는 역방향 graph 다

거버넌스



Sleep cadence 는 queue value 와 staleness SLO 로 정한다

시스템



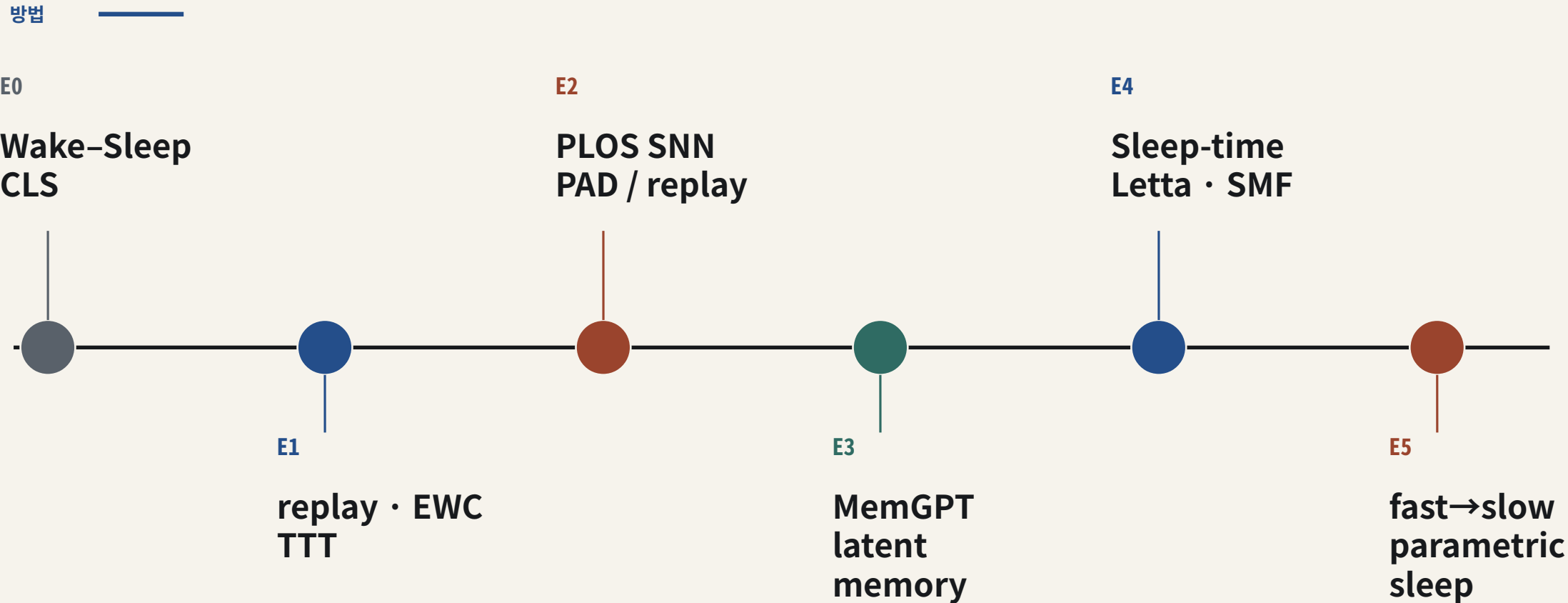
TRIGGER

idle·deadline·queue
depth·risk event

- TRIGGER
- BATCH
- BUDGET
- PREEMPT

idle·deadline·queue depth·risk event

Evidence 는 이름이 아니라 성숙도 ladder 로 읽는다



E0

Expected value 가 커도 evidence maturity 가 자동으로 높아지지 않는다 .

2025 STC 는 ‘질의 전 reasoning’ 의 직접 이득을 보였다

직접 근거

 $\approx 5 \times$

LATENCY



+13

LATENCY

6 deployable $\times$ 3 reuse $\times$ 3 pressure $\times$
3 seeds = 162
+ oracle on 27 operating cells

+18

GSM

2 schedules $\times$ 2 operators $\times$ 3 reuse
 $\times$ 3 pressure $\times$ 3 seeds

 $2.5 \times$

fixed blocks
16 families $\times$
3 seeds

 $\approx 5 \times$


global rows
104
continuous +
27 RF

+13%
max

blocked paired
bootstrap

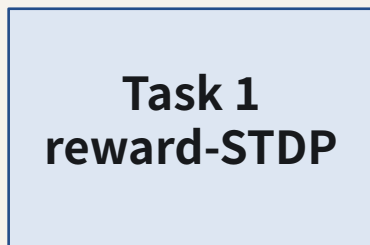
+18%
max

질의당 평균 비용

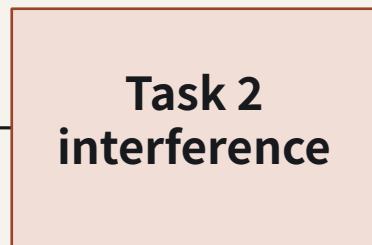
2.5 $\times$ ↓

2026 ‘Language Models Need Sleep’ 은 parametric sleep 을 구성했다

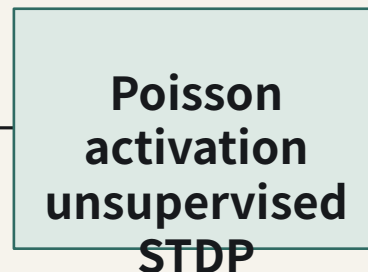
EXPAND



SEED



DREAM



EVAL



01

neurons
2 tasks
≥10 inits

EXPAND

02

SEED

03

LIMIT

old trace 가 완전히 사라진 뒤에는
복원하지 못했다 .
LLM · 다과제 · 장기용량 ·
deletion 은 검증하지 않았다 .

2026 continual fact study 는 weight write 의 reachability 를 반박...

05

adversarial findings

statistics
capacity
systems
publication

BREADTH

→ study data 가 bare statement 보다 application/com...

PLATEAU

→ sequential write 뒤 old-fact survival 이 낮은 수준에...

MISROUTE

→ 잊힌 응답의 다수가 최신 fact 로 routing

RECOVER

→ old fact 를 context 에 넣으면 77-80% 회복

IMPLICATION

→ stored 와 behaviorally reachable 은 다름

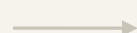
Weight 에 흔적이 남았다는 것만으로 usable long-term memory 가 아니다 .

Memory Caching 은 RNN state reuse 와 growing capacity 를 연결한다

CHECKPOINT



CHECKPOINT



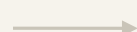
CHECKPOINT

Subquadratic
model 도 오래
살려면 state
cache policy 가
필요하다 .

AGGREGATE



AGGREGATE



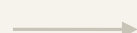
AGGREGATE

Subquadratic
model 도 오래
살려면 state
cache policy 가
필요하다 .

CAPACITY



CAPACITY



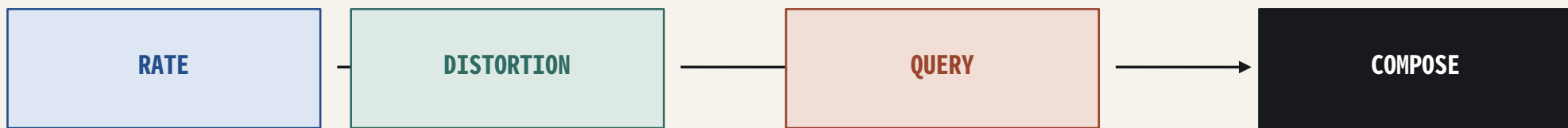
CAPACITY

Subquadratic
model 도 오래
살려면 state
cache policy 가
필요하다 .

segment 끝 recurrent memory state 저장

Rate-distortion 은 모든 memory layer 의 공통 한계를 준다

직접 근거 —



$$R \geq I^*(Q) \text{ or } \frac{\text{byte} \cdot \text{token} \cdot \text{state}}{D > 0 \quad \text{dimension} \cdot \text{factor}}$$

RATE

Budget 아래에서는 모든 것을 기억할 수 없고 query uncertainty 가 최저 비용을 정한다 .

원 history 대비 downstream output 변화

LoRA memory 에도 finite capacity 와 saturation knee 가 있다



RANK

- RANK
- DATA
- MODULE
- SYSTEM
- HYBRID

Adapter 는 유망한 compiled cache 이지
무한한 truth store 가 아니다 .

DANN 의 강한 문구는 현재 낮은 증거 등급으로 둔다

경계

05

adversarial findings

statistics
capacity
systems
publication

CLAIM

→ synthetic sleep · zero forgetting · vision/NLP

ACCESS

→ 동결 시점 full text · code · tables
검증 불가

RISK

→ abstract 수치를 direct evidence
처럼 인용할 수 없음

USE

→ hypothesis generator 로만 유지

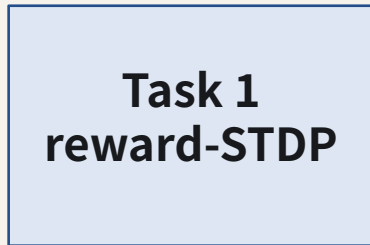
UPGRADE

→ full artifact · baseline · cycle curve
공개 필요

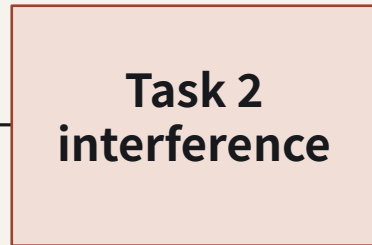
‘Zero forgetting’은 검증 가능한 repeated-cycle trace 가 나올 때까지 결론이
아니다 .

OpenAI Dreaming 은 strict external lifecycle STC 의 제품 신호다

INPUT



BACKGROUND



OUTPUT



CONTROL



01

neurons
2 tasks
≥10 inits

INPUT

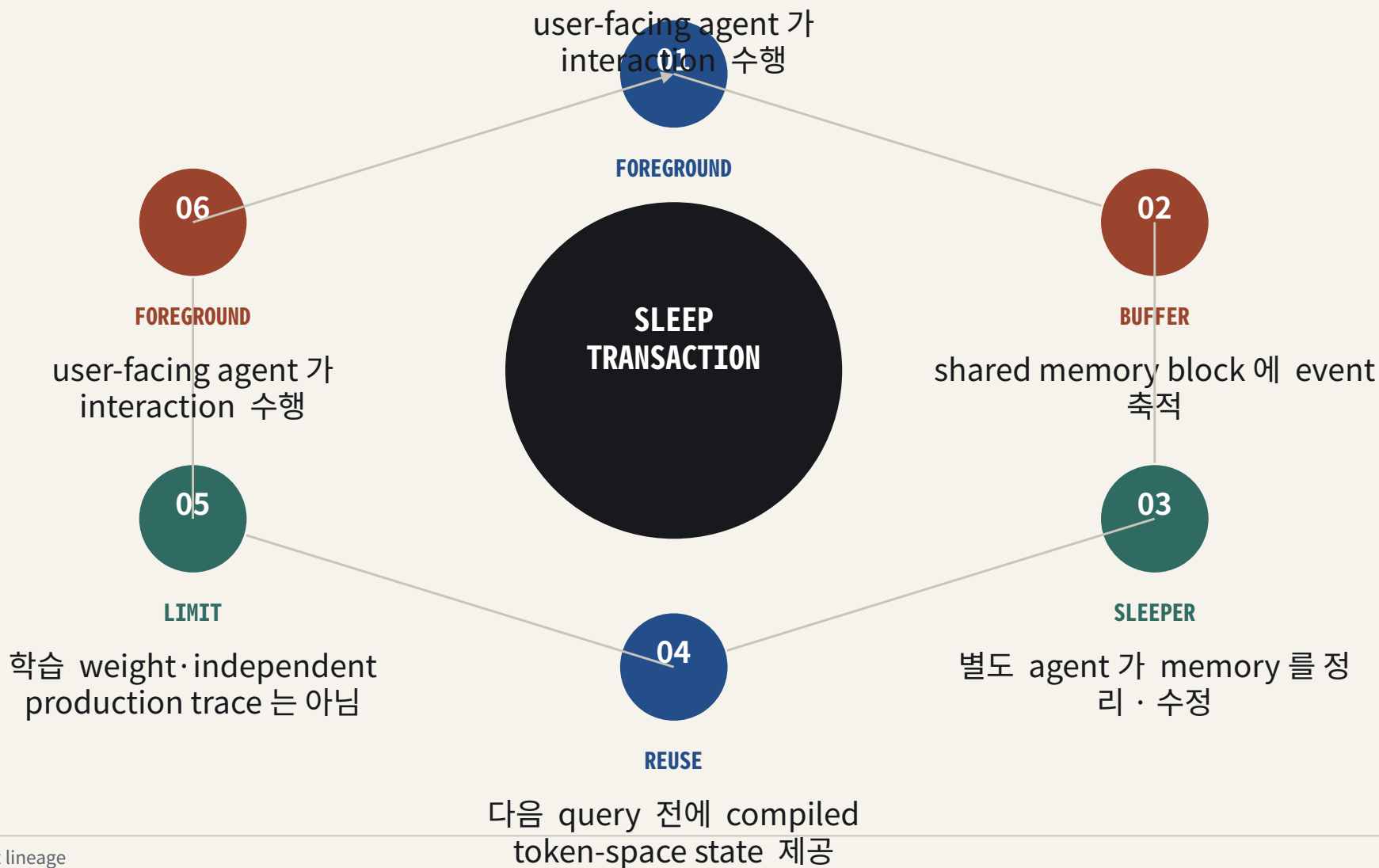
02

BACKGROUND 03

LIMIT

old trace 가 완전히 사라진 뒤에는
복원하지 못했다 .
LLM · 다과제 · 장기용량 ·
deletion 은 검증하지 않았다 .

Letta 는 sleeptime agent 를 pre-query memory compiler 로 구현한다



Mem0 는 extraction · update · graph 를 background memory 로 묶는다

제품 / 수를
관리

EXTRACT · conversation 에서 candidate memory 생성
DECIDE · add · update · delete · no-op
STORE · vector/graph 와 metadata 갱신
SERVE · query-time retrieval 로 context 축소
EVIDENCE · 제품 · benchmark 신호 ; weight consolidation 아님
EXTRACT · conversation 에서 candidate memory 생성
DECIDE · add · update · delete · no-op

EXTRACT

DECIDE

STORE

SERVE

EVIDENCE

EXTRACT

DECIDE

EXTRACT · conversation 에서 candidate memory 생성

DECIDE · add · update · delete · no-op

STORE · vector/graph 와 metadata 갱신

SERVE · query-time retrieval 로 context 축소

EVIDENCE · 제품 · benchmark 신호 ; weight consolidation 아님

EXTRACT · conversation 에서 candidate memory 생성

DECIDE · add · update · delete · no-op

EXTRACT · conversation 에서 candidate memory 생성

DECIDE · add · update · delete · no-op

STORE · vector/graph 와 metadata 갱신

SERVE · query-time retrieval 로 context 축소

EVIDENCE · 제품 · benchmark 신호 ; weight consolidation 아님

EXTRACT · conversation 에서 candidate memory 생성

DECIDE · add · update · delete · no-op

EXTRACT · conversation 에서 candidate memory 생성

DECIDE · add · update · delete · no-op

STORE · vector/graph 와 metadata 갱신

SERVE · query-time retrieval 로 context 축소

EVIDENCE · 제품 · benchmark 신호 ; weight consolidation 아님

EXTRACT · conversation 에서 candidate memory 생성

DECIDE · add · update · delete · no-op

conversation 에서 candidate memory 생성

Zep 은 memory 를 temporal truth-maintenance 문제로 본다

논문 / 제품
근거

EPISODE

원 대화 · event 를 보존

ENTITY

stable node 와 semantic relation

TIME

validity 와 chronology

CONFLICT

새 assertion 이 old 를 supersede

RETRIEVE

current-state 와
historical provenance
를 함께 제공

원 대화 · event 를 보존

Google 은 parametric sleep 과 external self-evolution 을 병렬 탐색한다



Knowledge Seeding · Dreaming 의 offline adaptation

Meta 는 explicit memory 와 self-editable agent program 을 연구한다

PAHF

PAHF

PAHF

Self-improving agent 와 sleep lifecycle 은 겹치지만 동일하지 않다 .

HYPERAGENT

HYPERAGENT

HYPERAGENT

Self-improving agent 와 sleep lifecycle 은 겹치지만 동일하지 않다 .

SIGNAL

SIGNAL

SIGNAL

Self-improving agent 와 sleep lifecycle 은 겹치지만 동일하지 않다 .

clarification · feedback 을 per-user external memory 에 반영

Microsoft 는 destination-neutral memory benchmark 를 강화한다

EXTERNAL



EXTERNAL



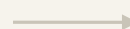
EXTERNAL

업계는 ‘기억이 있는가’ 보다 실제 task value 를 측정하는 방향으로 이동한다 .

PARAMETRIC



PARAMETRIC



PARAMETRIC

업계는 ‘기억이 있는가’ 보다 실제 task value 를 측정하는 방향으로 이동한다 .

BENCH



BENCH



BENCH

업계는 ‘기억이 있는가’ 보다 실제 task value 를 측정하는 방향으로 이동한다 .

LongMem 의 frozen backbone+side memory

학계는 네 계보가 lifecycle 관점에서 수렴한다

통합 추론

REPLAY

Wake-Sleep
CLS

NEURAL MEMORY

PLOS SNN
PAD / replay

REPLAY

Sleep-time
Letta · SMF

PLASTICITY

replay · EWC
TTT

COMPACTION

MemGPT
latent
memory

PLASTICITY

fast→slow
parametric
sleep

REPLAY

주류가 되는 것은 하나의 sleep algorithm 보다 lifecycle composition 일 가능성이 높다 .

여섯 핵심 주장 중 강한 결론은 두 개뿐이다

조건부 결론

NEW
AXIS
·
질의 전 / 후
reusable
MAINSTREAM
·
computation
·
likely, 명칭은
CONTEXT
·
reuse workload
·
CONTINUAL
·
forgetting
PARAMETRIC
·
external
·
보다
·
보편
·
우월하지
·
않음

NEW AXIS
MAINSTREAM
LONG CONTEXT
CONTINUAL
PARAMETRIC

질의 전 / 후 reusable computation
은 실재

lifecycle 은 likely, 명칭은
uncertain

특정 reuse workload 에서만 우월

보조하지만 forgetting 해결 증거 부족

external 보다 보편 우월하지 않음



NEW AXIS

가치는 reuse·stability·verifiability 가 높은 workload 에 집중된다

	CODEBASE · 반복 query·stable repository → 높음	PERSONAL · high reuse 이나 privacy/volatility gate → 중간	ENTERPRISE · evidence·delete 요구가 높아 external 우세	NEWS · volatile truth → parametric 낮음
CODEBASE · 반복 query·stable repository → 높음	CODEBASE	PERSONAL	ENTERPRISE	
PERSONAL · high reuse 이나 privacy/volatility gate → 중간	Letta sleeper Auto-Dreamer	Cartridges Offline Recurrence	CODEBASE	PERSONAL
ENTERPRISE · evidence·delete 요구가 높아 external 우세	memory-policy pretraining		NEWS	ONE-SHOT

반복 query·stable repository → 높음

high reuse 이나 privacy/volatility gate → 중간

주류화 여부는 다섯 시나리오로 추적한다

전망

EXTERNAL DEFAULT

- EXTERNAL DEFAULT
- HYBRID
- PARAMETRIC NICHE

HYBRID

- EXTERNAL DEFAULT
- HYBRID
- PARAMETRIC NICHE

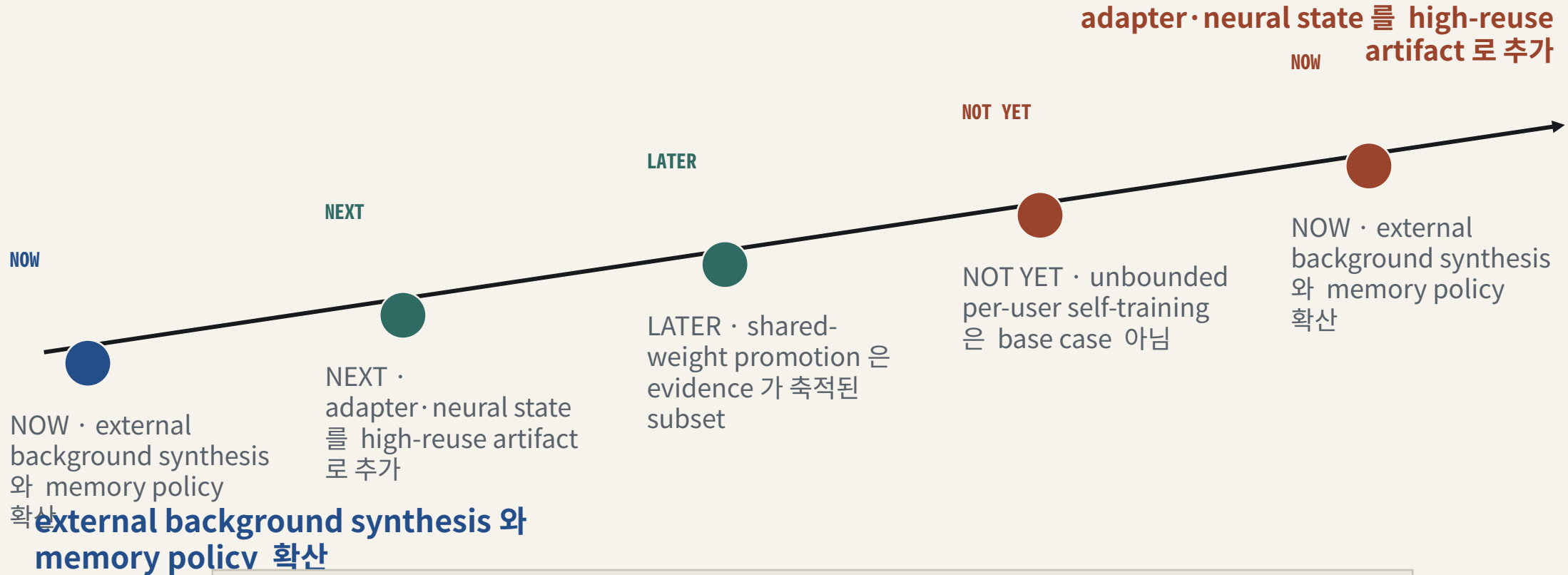
PARAMETRIC NICHE

- EXTERNAL DEFAULT
- HYBRID
- PARAMETRIC NICHE

EXTERNAL
DEFAULT

달력 예측 대신 각 시나리오의 leading indicator 와 falsifier 를 본다 .

가장 가능성 높은 경로는 lifecycle mainstream · parameter niche 다



이 축은 주류가 될 수 있지만 대부분 'sleep' 이라는 이름이나 full-weight training 형태는 아닐 것이다 .

Sleep scaling 은 compute 를 미래 query 에 선불 배분하는 법칙이다



$$C = C_{\text{wake}} + \frac{C_{\text{sleep}}}{N_{\text{reuse}}} \longrightarrow \text{query 전 transform} \cdot \text{training budget}$$

SLEEP

총 FLOP 보다 누가 언제 기다리고 몇 번 재사용하는지가 새 축을 만든다 .

query 가 도착한 뒤 latency-critical budget

순가치는 quality gain 에서 다섯 비용을 뺀 값이다

가설



$$V = E_q[\Delta U_q] - \lambda C_{\text{build}} - \text{future query 분포에서의 expected}$$

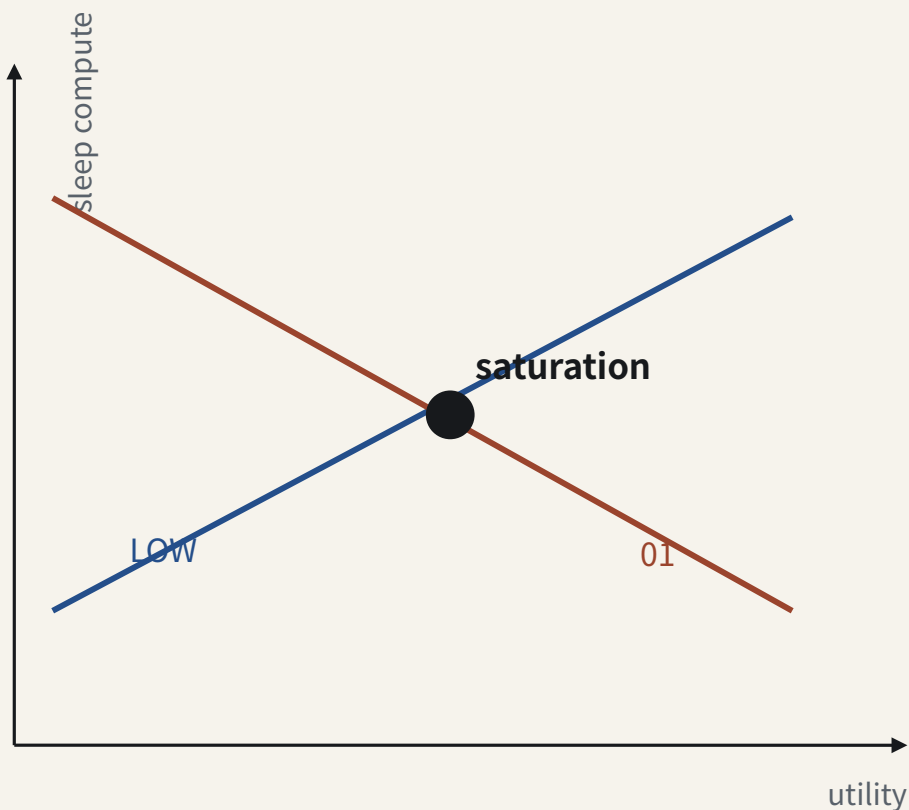
VALUE

Sleep budget 은 gain 이 아니라 lifecycle net value 로 결정한다 .

generate · train · compact 비용

Compute scaling 은 초기에 이득이 크고 뒤에는 포화한다

검증 전 가설



LOW

MID

HIGH

SHIFT

LOW

— coverage 부족 : 중요한 relation 미계산

— high-probability query 를 빠르게 precompute

— 중복 · irrelevant dream · verification debt 증가

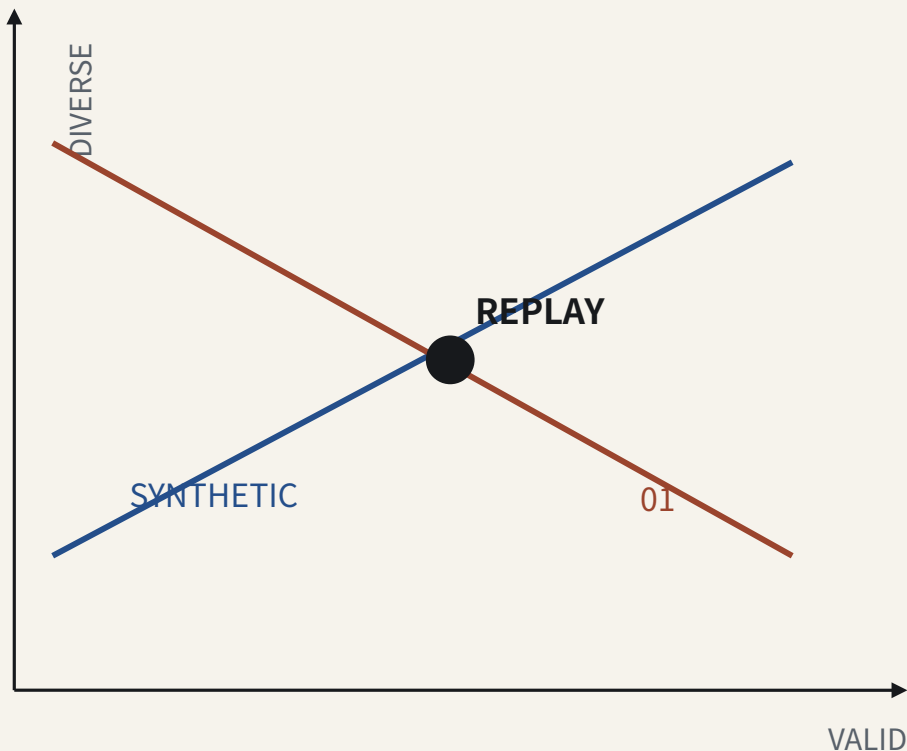
— better policy 가 curve 를 위 / 왼쪽으로 이동

— coverage 부족 : 중요한 relation 미계산

무제한 sleep FLOP 이 아니라 policy-conditioned diminishing return 을 예상한다 .

Data scaling 의 독립변수는 sample 수가 아니라 valid diversity

검증 전 가설



VALID

grounded · consented
· correct outcome

DIVERSE

task · time · failure
mode coverage

REPLAY

old capability 를
대표하는 support

SYNTHETIC

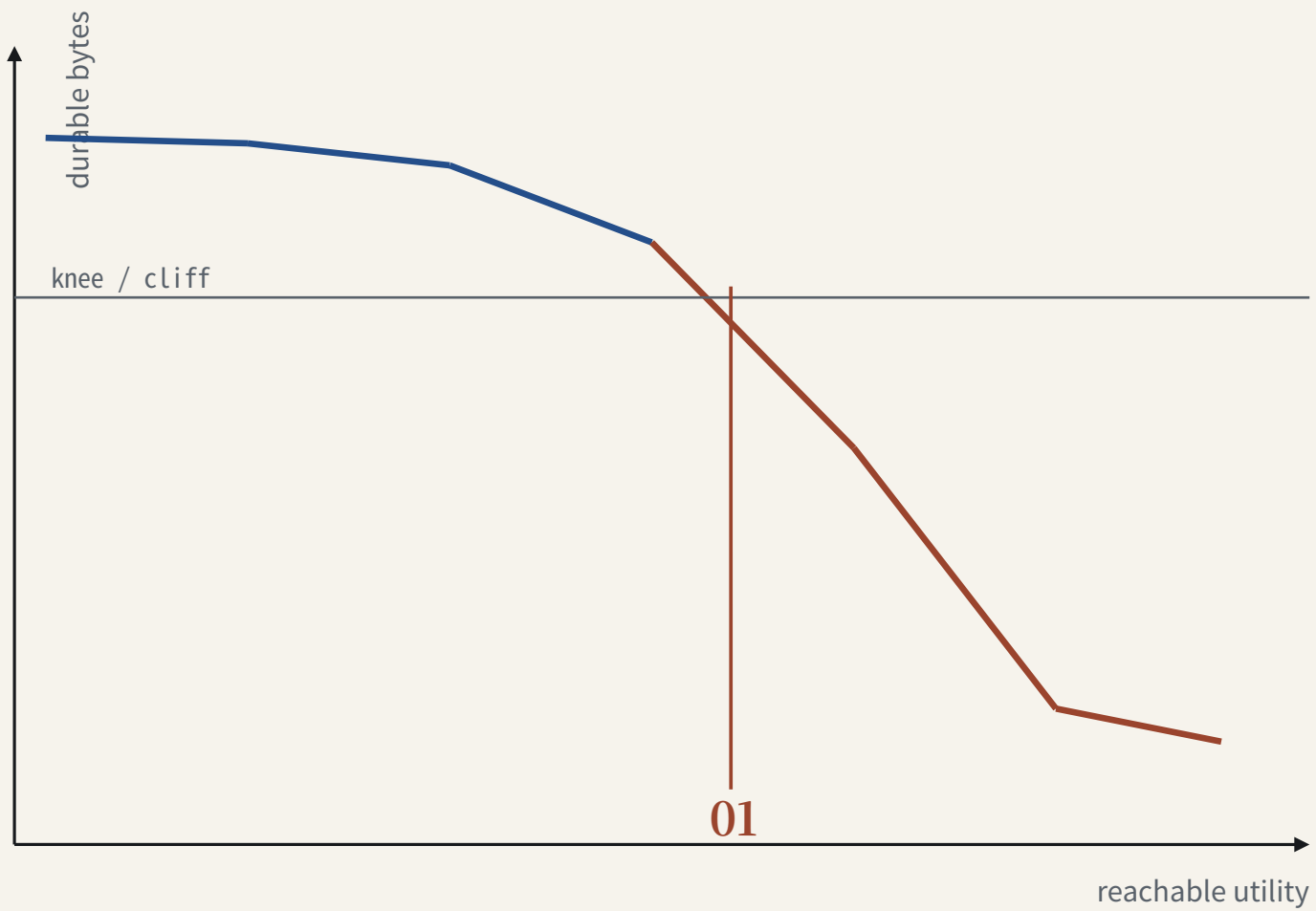
real support 를 넓히는
candidate

POISON

오류율이 durable harm
으로 누적

같은 token budget 에서도 provenance 와 coverage
가 effective data 를 결정한다 .

Capacity scaling 에는 utility knee 와 interference cliff 가 있다



GROW

- GROW

- KNEE

- CLIFF

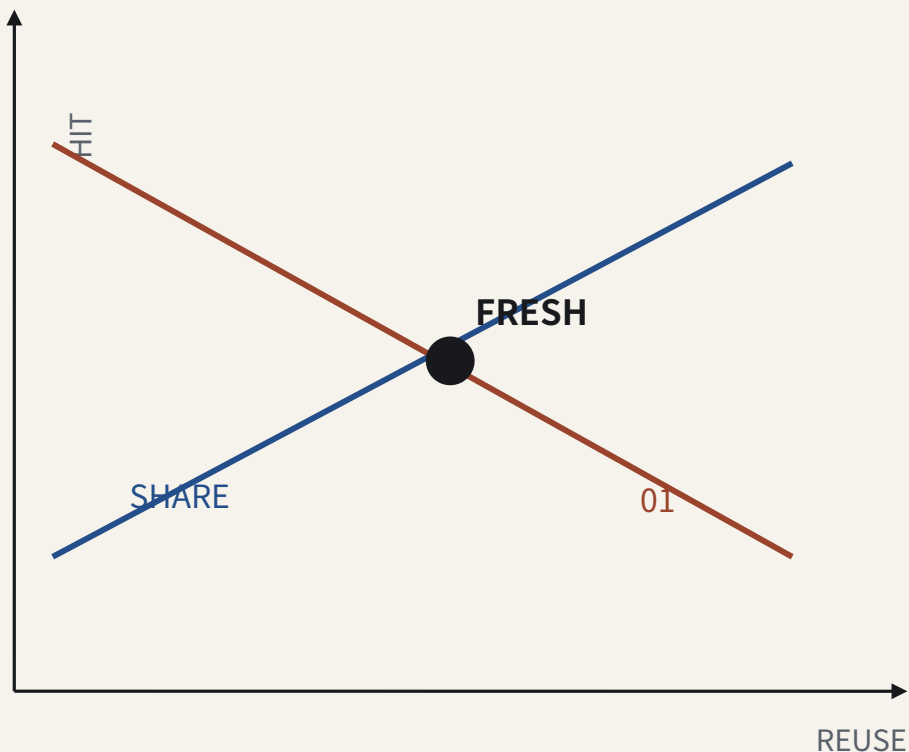
- RESET

- GROW

용량은 parameter count 가 아니라
reachable utility per byte 로 측정한
다.

Reuse 와 predictability 가 sleep economics 의 곱셈항이다

검증 전 가설



REUSE

HIT

FRESH

SHARE

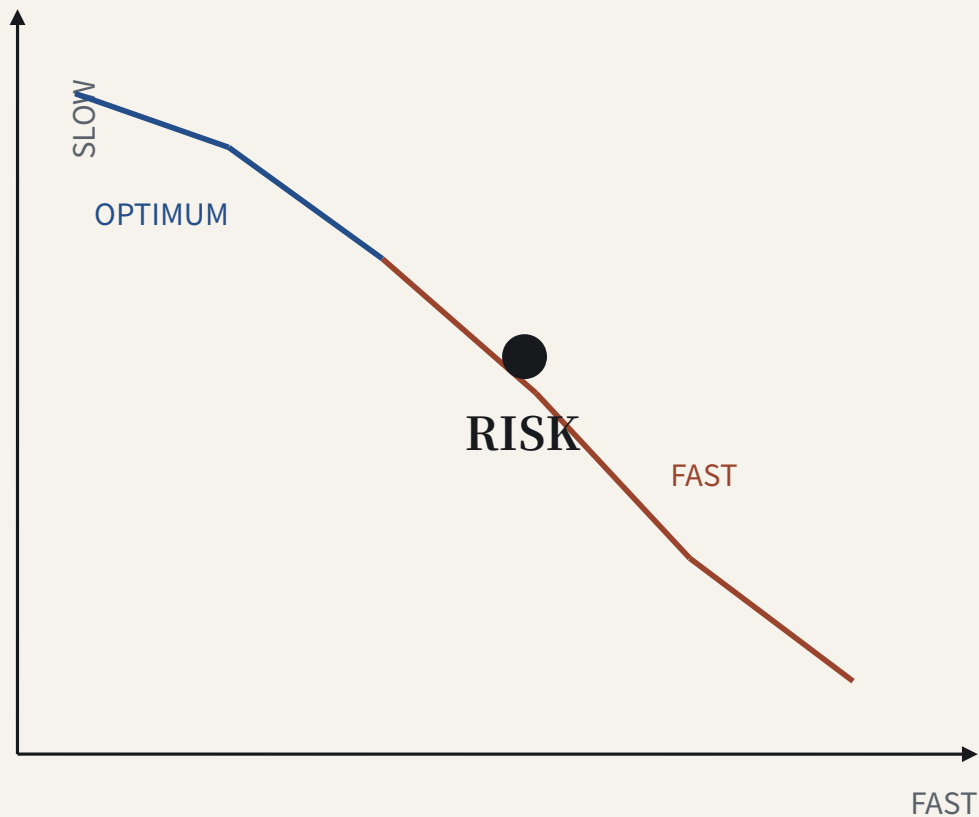
REUSE

- 같은 context/domain state 가 다시 조회되는 횟수
- precomputed artifact 가 실제 query 에 유용한 확률
- artifact 가 invalidation 전 유효한 기간
- 여러 tenant/query 가 안전하게 공동 사용
- 같은 context/domain state 가 다시 조회되는 횟수

One-shot · volatile · unpredictable workload 에서는 sleep 이 구조적으로 불리하다 .

Cadence scaling 은 staleness 와 batch efficiency 를 교환한다

검증 전 가설 ———



SLOW

$$T^* = \operatorname{argmin}_T C_{\text{batch}}(T) + C_{\text{stale}}(T) + C_{\text{risk}}(T)$$

- SLOW
- RISK
- OPTIMUM

fresh 하지만 작은 batch · 높은 launch/verify overhead

Robustness cost 를 빼면 scaling curve 는 허상이다

Red team

05

adversarial findings

statistics
capacity
systems
publication

RETENTION

→ old capability regression

COLLAPSE

→ recursive synthetic-data tail loss

POISON

→ persistent backdoor · prompt injection

PRIVACY

→ derived state 의 exposure · cross-tenant leak

ERASURE

→ delete propagation 과 residual proof

안전 · 삭제 · 회귀 검증은 optional overhead 가 아니라 scaling denominator 다 .

External ↔ parametric crossover 는 reuse threshold 로 계산한다



$$N^* = \frac{(C_{\text{train}} + C_{\text{verify}} + C_{\text{update}})}{(C_{\text{build}} + C_{\text{query}} + C_{\text{retrieval}} + C_{\text{prefilter}})} \longrightarrow \text{낮은 build, query 마다 retrieval/prefilter}$$

EXTERNAL

Adapter 승격은 stable high-reuse subset 에서만 경제성이 생긴다 .

높은 train/verify, 낮은 반복 serve 비용

Scaling law 를 찾으려면 여섯 축을 독립 sweep 한다

실험 설계

	COMPUTE · sleep FLOP · token · wall-time	DATA · valid diversity · replay ratio	CAPACITY · bytes · rank · modules · facts	REUSE · queries/context · hit probability
COMPUTE · sleep FLOP · token · wall-time	COMPUTE	DATA	CAPACITY	
DATA · valid diversity · replay ratio	Letta sleeper Auto-Dreamer	Cartridges Offline Recurrence	RISK	COMPUTE
CAPACITY · bytes · rank · modules · facts	memory-policy pretraining		CAPACITY	REUSE

sleep
FLOP · to
ken · wall
-time

valid diversity · replay ratio

현재 scaling law 는 결과가 아니라 검증 가능한 hypothesis set 이다

근거 장부

01	H1 · compute diminishing return	08	STC-H002
02	H2 · valid-diversity data law	09	STC-H003
03	H3 · capacity knee/cliff	10	STC-H004
04	H4 · reuse × predictability amortization	11	STC-H005
05	H5 · cadence–staleness optimum	12	STC-H001
06	STATUS · all prospective; no universal exponent claimed	13	STC-H002
07	STC-H001	14	STC-H003

compute diminishing return

다음 연구는 exponent 보다 falsifier 를 먼저 공개한다

연구 질문

HIT

- HIT
- CYCLE
- DELETE

CYCLE

- HIT
- CYCLE
- DELETE

DELETE

- HIT
- CYCLE
- DELETE

HIT

하나라도 실패하면 broad adoption 주장을 좁힌다 .

Infra 는 wake·sleep·memory fabric 의 세 plane 으로 분리된다

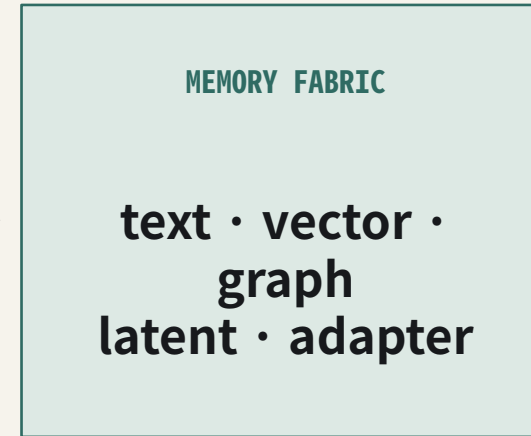
Plane 은 분리하되 artifact 와 version contract 로 연결한다 .



01



SLEEP PLANE



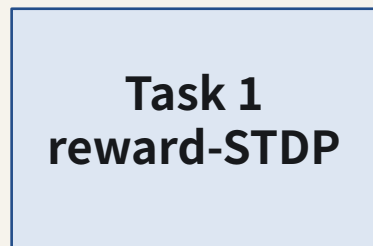
MEMORY FABRIC

RESOURCE
LEDGER

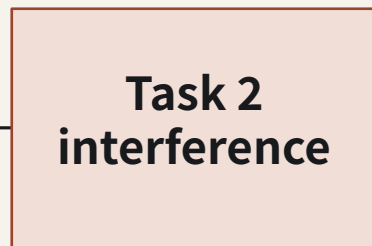
Wake cluster 는 응답과 evidence capture 를 끝까지 분리한다

설계

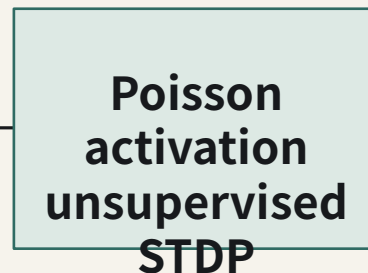
READ



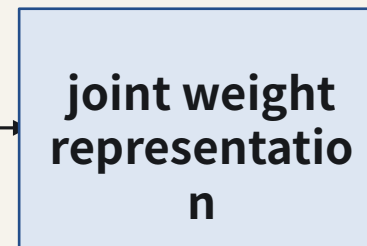
INFER



CAPTURE



ACK



01

neurons
2 tasks
≥10 inits

READ

02

INFER

03

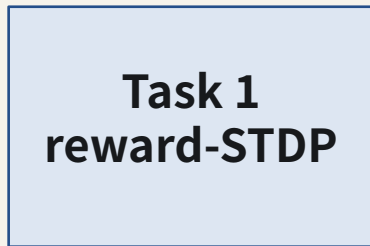
LIMIT

old trace 가 완전히 사라진 뒤에는
복원하지 못했다.
LLM · 다과제 · 장기용량 ·
deletion 은 검증하지 않았다.

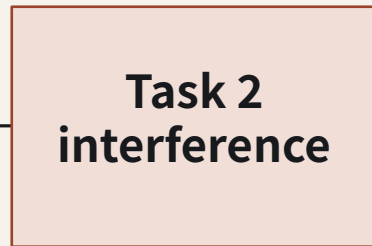
Sleep cluster 는 mixed inference+training workload 다

설계

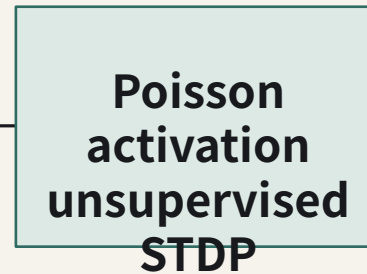
GENERATE



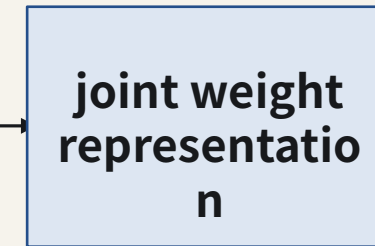
TRAIN



COMPACT



VERIFY



01

neurons
2 tasks
≥10 inits

GENERATE

02

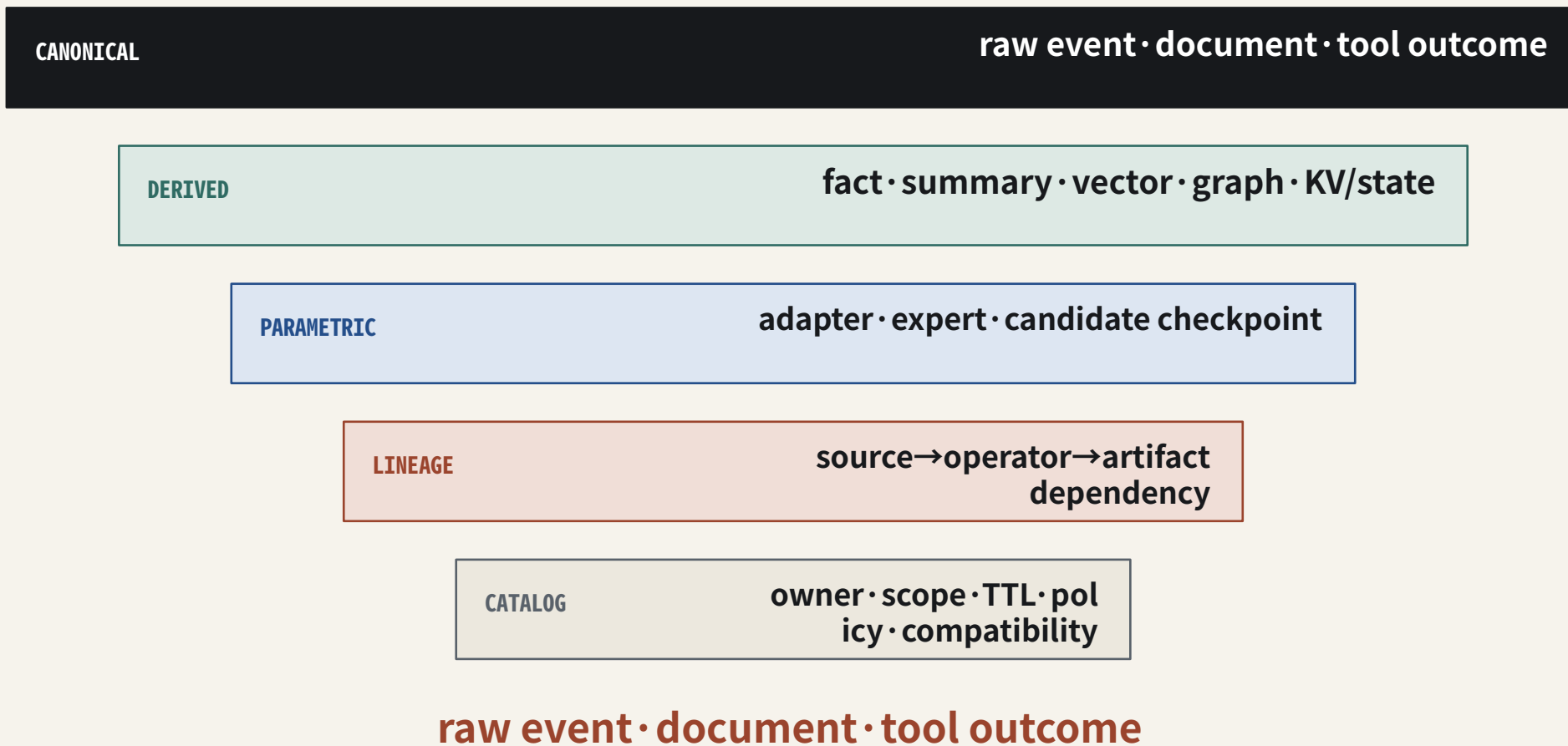
TRAIN

03

LIMIT

old trace 가 완전히 사라진 뒤에는
복원하지 못했다.
LLM · 다과제 · 장기용량 ·
deletion 은 검증하지 않았다.

Memory fabric 는 truth 와 compiled artifact 를 같은 graph 로 잇는다

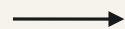


Data movement 는 state locality 와 delta transport 로 줄인다

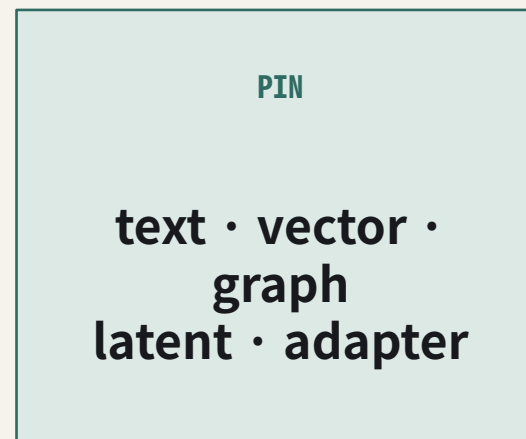
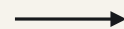
Wake↔sleep 간 전체 state 복사는 scaling law 를 data-movement law 로 바꾼다 .



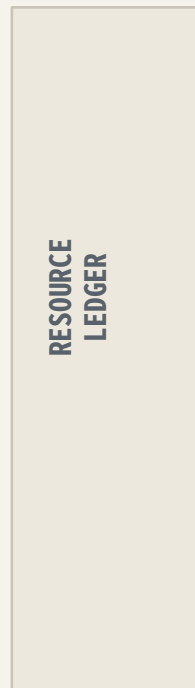
01



DELTA

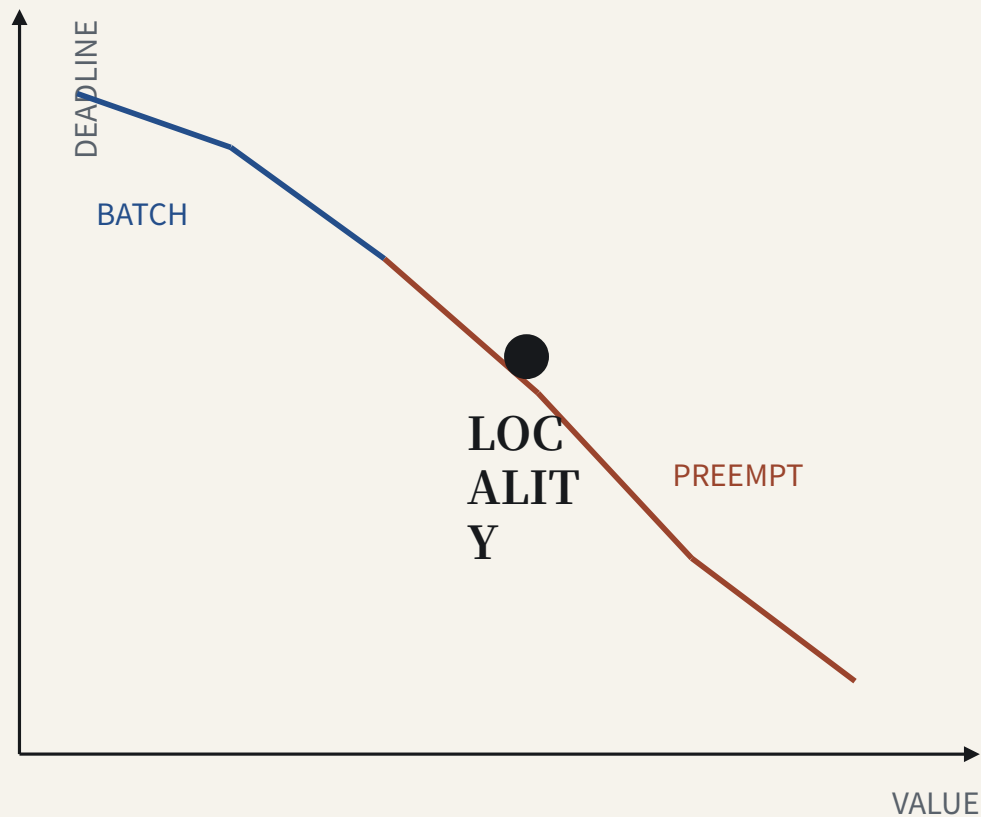


PIN



Scheduler 는 가치 · deadline · locality 를 함께 최적화한다

설계



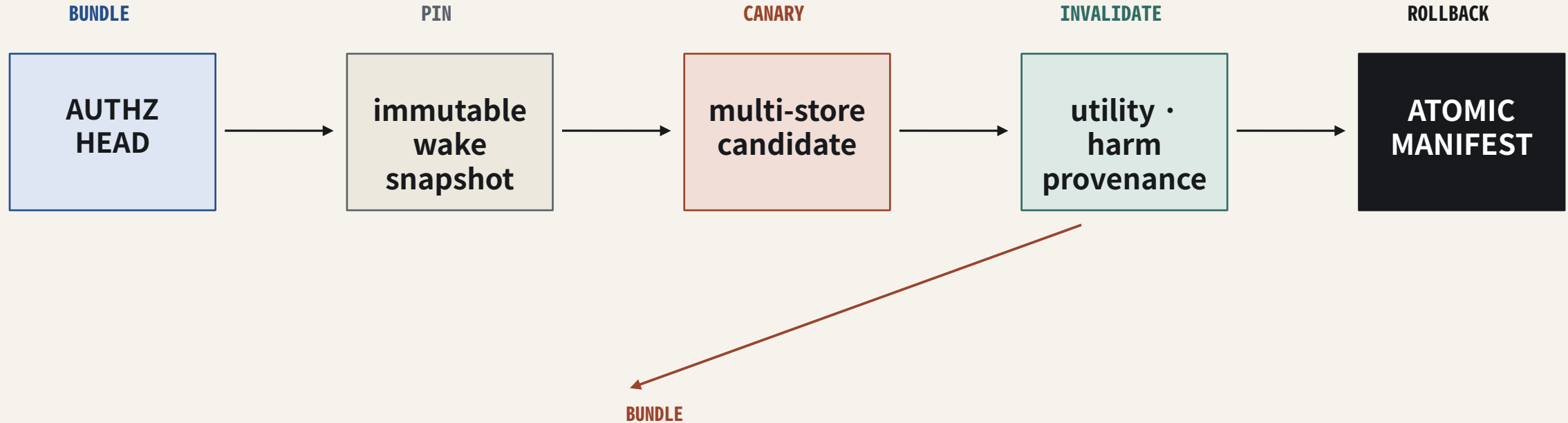
VALUE

expected future utility
per joule

- VALUE
- DEADLINE
- LOCALITY
- BATCH

expected future utility per joule

Snapshot version 은 model · memory · index 를 하나의 read view 로 묶는다



부분 publish 는 stale index 와 새 adapter 가 섞이는 silent failure 를 만든다 .

부분 publish 는 stale index 와 새 adapter 가 섞이는 silent failure 를 만든다 .

Persistent memory 는 새로운 control-plane attack surface

다보안

05

adversarial findings

statistics
capacity
systems
publication

INGEST

→ prompt injection 이 replay candidate 로 승격

TRAIN

→ poison 이 adapter/weight 에 durable imprint

RETRIEVE

→ cross-tenant · secret exfiltration

PUBLISH

→ malicious artifact · rollback blocking

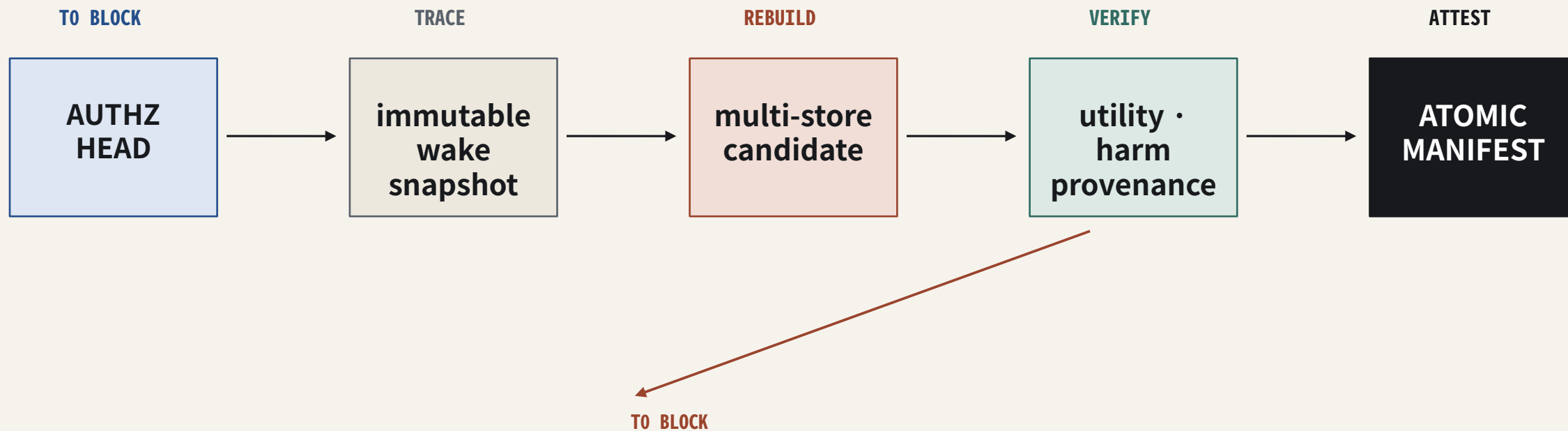
AUDIT

→ output-only test 가 hidden memory 를 놓침

Sleep worker 는 sandbox · signature · tenant isolation · provenance gate 가 필요하다 .

Deletion SLA 는 serving 차단과 physical rebuild 를 분리한다

거버넌스



빠른 logical delete 와 느린 compaction 을 하나의 모호한 약속으로 만들지 않는다 .

빠른 logical delete 와 느린 compaction 을 하나의 모호한 약속으로 만들지 않는다 .

관측성은 FLOP 보다 lifecycle accounting 을 먼저 보여야 한다

계측

 ΔU

QUALITY



J/

QUALITY

6 deployable $\times$ 3 reuse $\times$ 3 pressure $\times$
3 seeds = 162
+ oracle on 27 operating cells

y

B/

COMPUTE

2 schedules $\times$ 2 operators $\times$ 3 reuse
 $\times$ 3 pressure $\times$ 3 seeds

y

p99

fixed blocks
16 families $\times$
3 seeds

 $\Delta U / \epsilon$

global rows
104
continuous +
27 RF

FLOP

blocked paired
bootstrap

bytes

raw/derived/
adapter
footprint

bytes

Device opportunity 는 destination 과 operator 의 교차점에 있다

	TEXT/GRAPH · small random I/O · metadata · snapshot	VECTOR/KV · high-read BW · quantize · tiering	ADAPTER · mixed read/write · optimizer · checkpoint	SHARED · large sequential training traffic
TEXT/GRAPH · small random I/O · metadata · snapshot	TEXT/GRAPH	VECTOR/KV	ADAPTER	
VECTOR/KV · high-read BW · quantize · tiering	Letta sleeper Auto-Dreamer	Cartridges Offline Recurrence	TEXT/GRAPH	VECTOR/KV
ADAPTER · mixed read/write · optimizer · checkpoint	memory-policy pretraining		SHARED	CONTROL
small random I/O · metadata · snapshot	high-read BW · quantize · tiering			

HBM 기회는 inference 와 training traffic 이 겹치는 구간이다

기회 ———



base
weight·teacher·replay
→
base
weight·teacher·replay

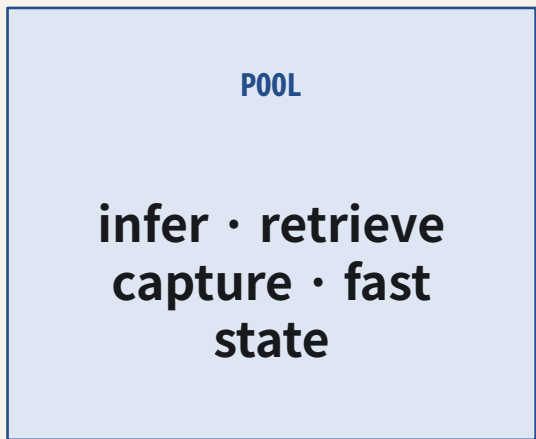
READ

Peak FLOP 보다 mixed-phase bandwidth 와 preemption recovery 가
차별점이 된다 .

gradient·optimizer·adapter delta·checkpoint

CXL·pooled memory 는 큰 mutable state 의 residency 문제를 푼다

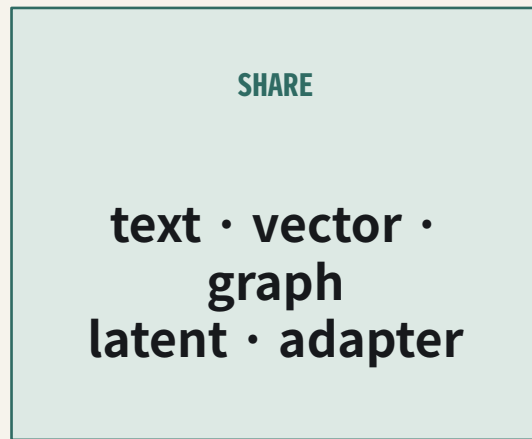
용량 이득이 locality 손실로 상쇄되지 않는 workload boundary 를 찾아야 한다 .



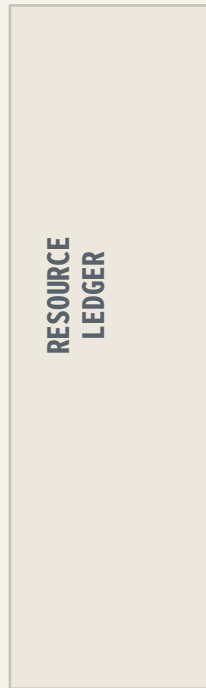
01



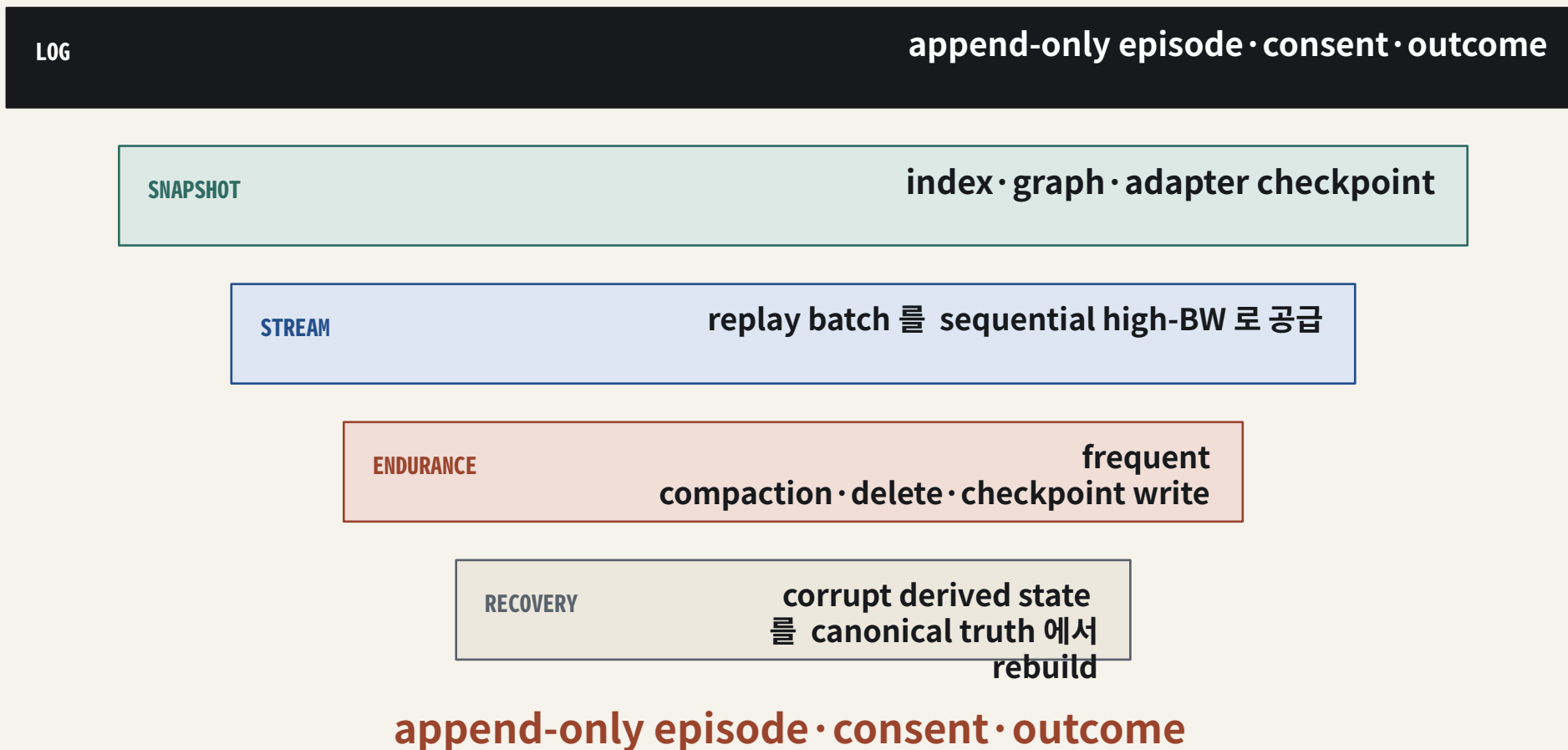
MIGRATE



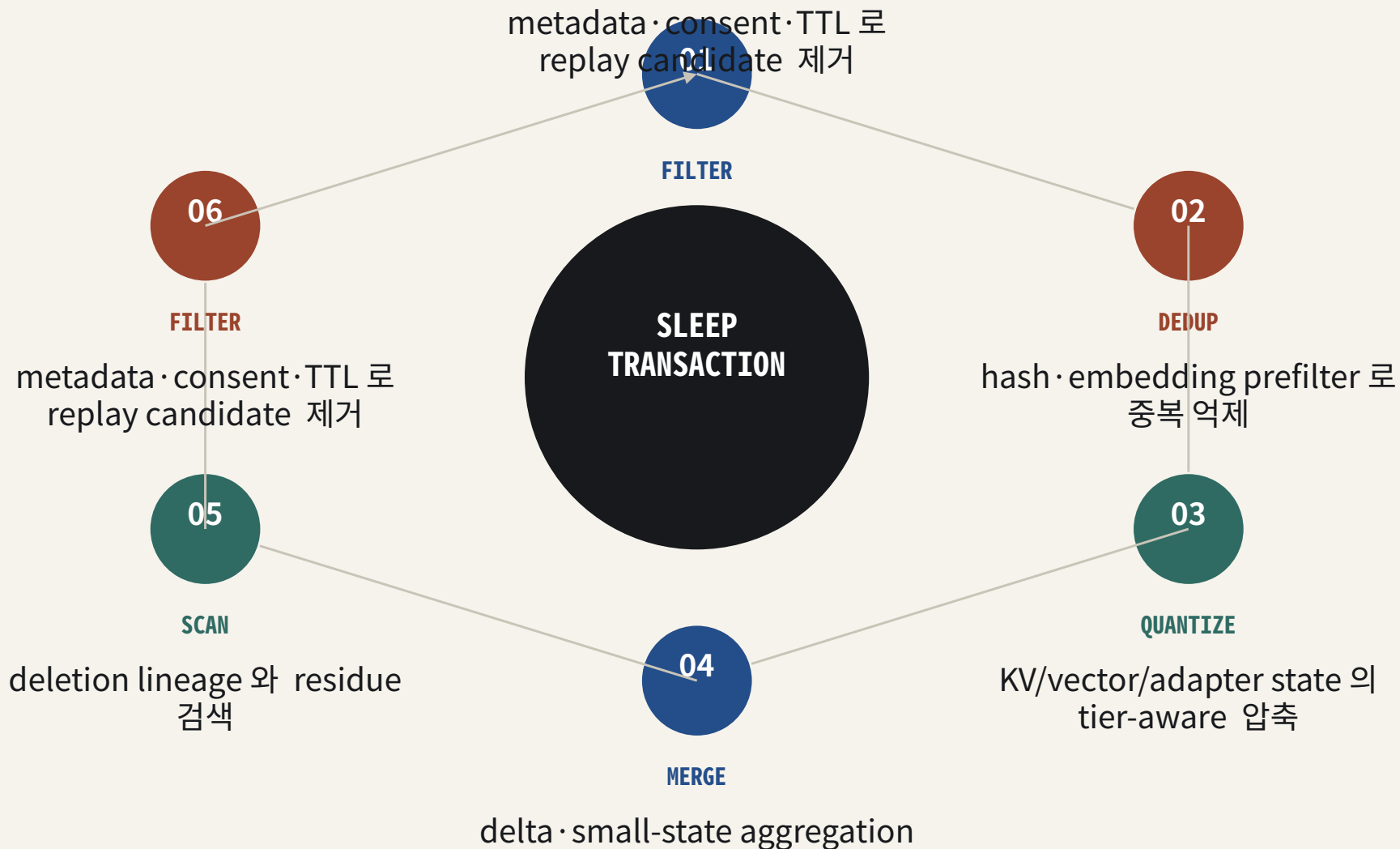
SHARE



SSD · object tier 는 canonical truth 와 rebuild bandwidth 를 담당한다



Near-memory operator 는 byte 를 줄이는 지점에서만 가치가 있다 —



Samsung 이 우선 검증할 여섯 실험은 명확하다

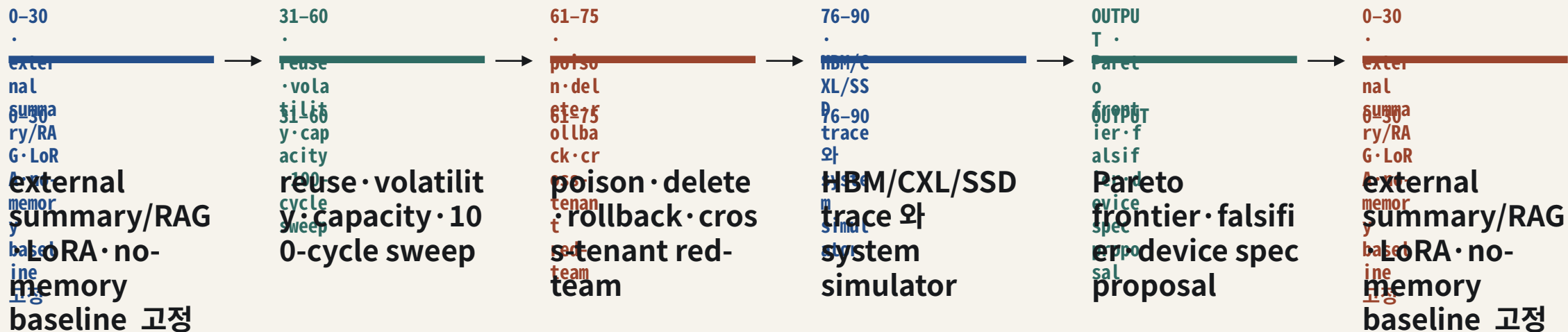
실험 장부

01	E1 HBM · generate/train/eval phase 별 bytes·joule	08	STC-S15
02	E2 PREEMPT · wake burst 중 checkpoint·resume overhead	09	STC-C047
03	E3 TIER · HBM↔CXL↔SSD hot-state policy 와 p99	10	STC-C048
04	E4 SNAPSHOT · full vs delta checkpoint write amplification	11	STC-S14
05	E5 CAPACITY · state bytes 대비 utility knee	12	STC-S15
06	E6 DELETE · lineage scan·rebuild·attestation cost	13	STC-C047
07	STC-S14	14	STC-C048

generate/train/eval phase 별 bytes·joule

90 일 benchmark 는 baseline · cycle · device 를 한 번에 묶는다

실행



0-30

같은 evidence stream 과 query trace 로 모든 destination 을 비교한다 .

01

결론 : sleep 은 모델 기능이 아니라 검증 가능한 memory economy 다

PROMISING

CONDITIONAL

LAW

INFRA



Wake 가 exact event 를 남기고
Sleep 이 candidate 를 만들며
Memory fabric 이 version 을 보유하고
Control plane 이 publish 와 undo 를 결정한다

background external transformation 은 이미 실용적